

C# DO JEITO CERTO

EleMar

AULA 6

INTEGRAÇÕES ASSÍNCRONAS



CAPÍTULO 1

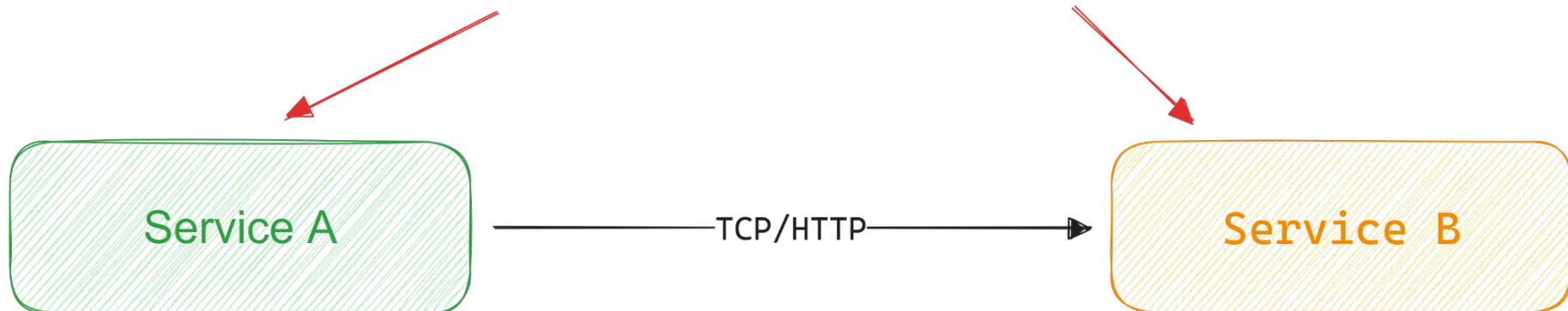
DESACOPLANDO E SUPERANDO LIMITAÇÕES DE INTEGRAÇÕES SÍNCRONAS





Limitações integrações síncronas

Qualquer indisponibilidade do Service B afeta o Service A



- **Acoplamento:** Falhas em algum serviço podem impactar todos os serviços, gerando um efeito cascata de indisponibilidade.
- **Disponibilidade:** Todos os serviços precisam estar operantes para que a operação seja bem sucedida.
- **Expansão:** Inclusão de novos serviços impacta a nível de implementação serviços existentes.
- **Tempo de resposta:** Cliente precisa aguardar a execução completa finalizar

CAPÍTULO 1

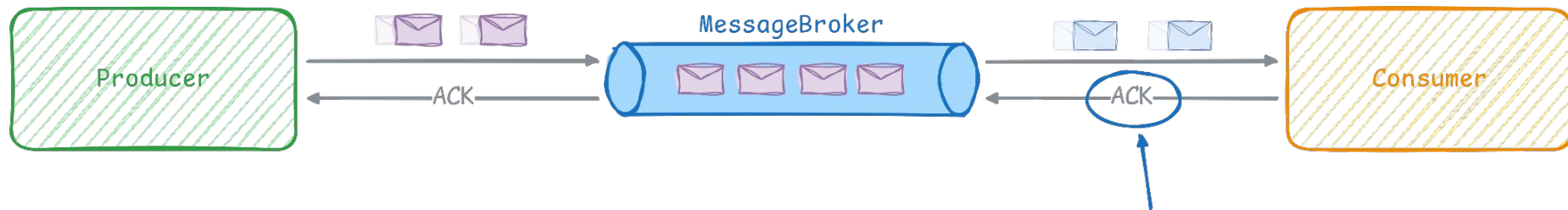
INTEGRAÇÃO ASSÍNCRONA



BENEFÍCIOS E VANTAGENS DA INTEGRAÇÃO ASSÍNCRONA

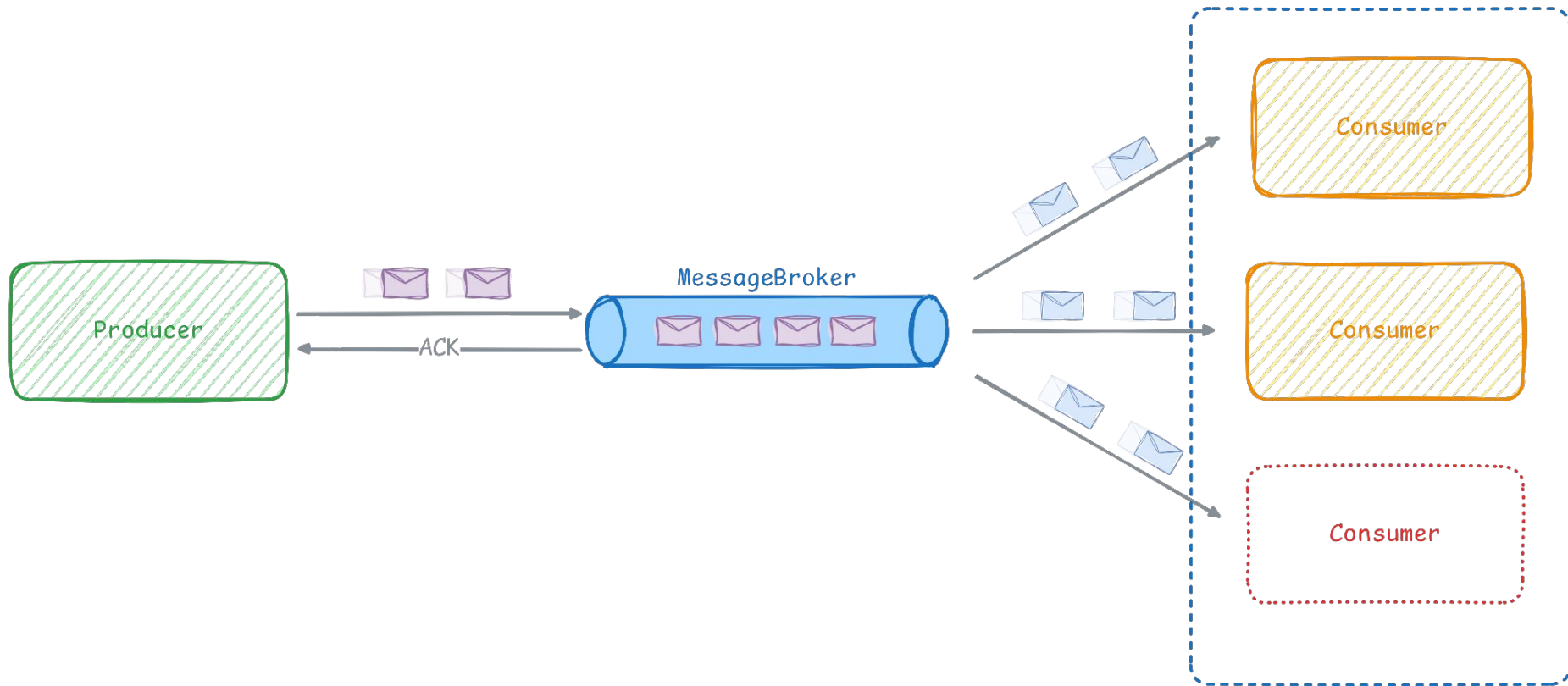
INTEGRAÇÕES ASSÍNCRONAS PERMITEM QUE SISTEMAS TROQUEM **EVENTOS E MENSAGENS** DE FORMA TOTALMENTE **DESACOPLADA, ESCALÁVEL E TOLERANTE A FALHAS**, GARANTINDO **ALTA DISPONIBILIDADE** E PROCESSAMENTO **PARALELO** COM EFICIÊNCIA.

COMUNICAÇÃO ASSÍNCRONA



Sinaliza para o messageBroker
que o evento foi processado com sucesso

COMUNICAÇÃO ASSÍNCRONA



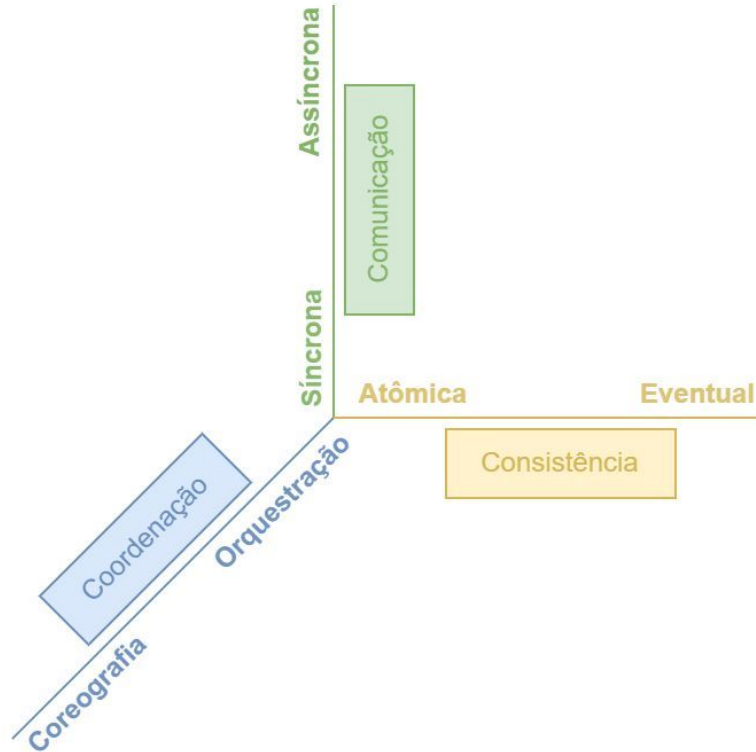
CAPÍTULO 2

GERENCIANDO SISTEMAS DISTRIBUÍDOS



3 EIXOS DO SISTEMA DISTRIBUÍDO

As 3 "forças" em sistemas distribuídos



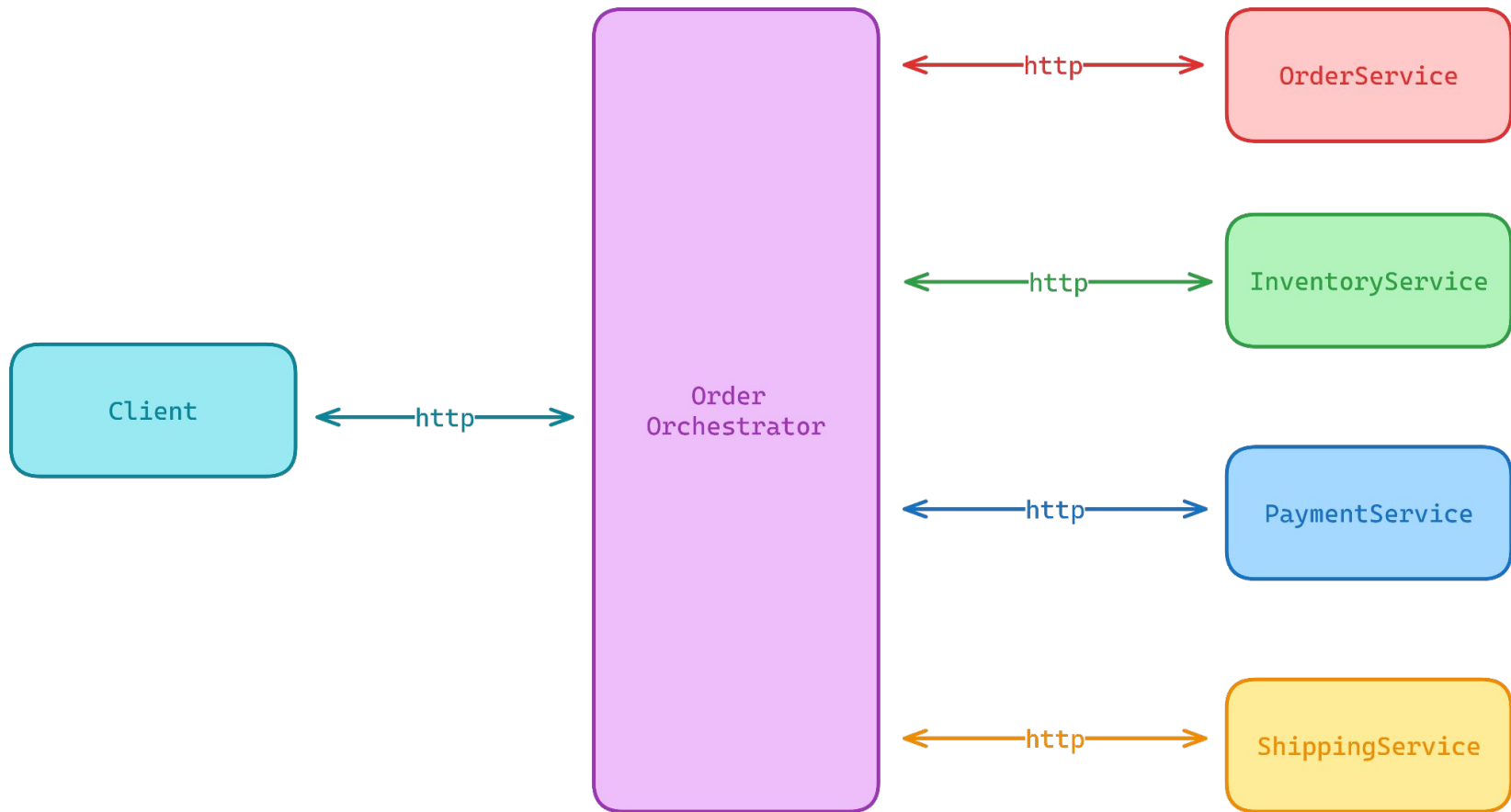
ORQUESTRAÇÃO

EleMar Jr

ORQUESTRAÇÃO

A **ORQUESTRAÇÃO** EM SISTEMAS DISTRIBUÍDOS **CENTRALIZA O CONTROLE DA EXECUÇÃO, DO ESTADO E DO TRATAMENTO DE ERROS** EM UM **ÚNICO** COMPONENTE, REDUZINDO O ACOPLAMENTO ENTRE SERVIÇOS E FACILITANDO A RECUPERAÇÃO DE FALHAS.

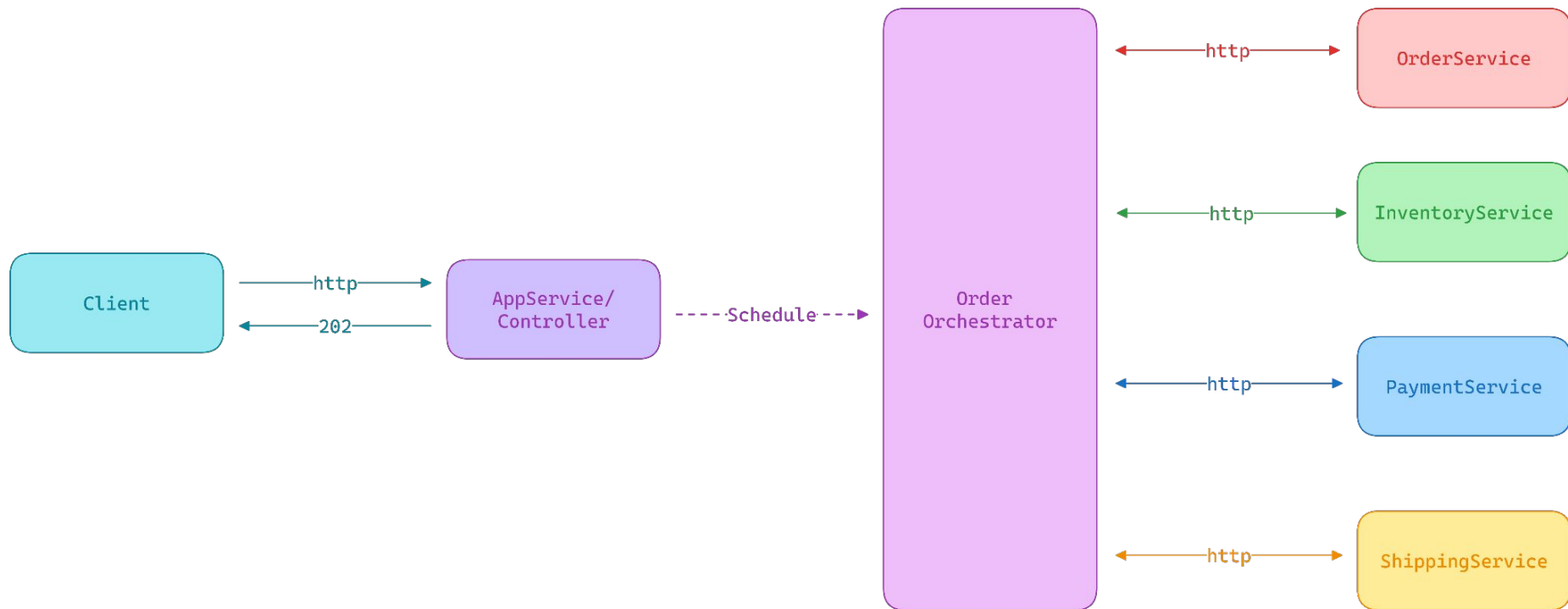
ORQUESTRAÇÃO SÍNCRONA



ORQUESTRAÇÃO - DESVANTAGENS

A **ORQUESTRAÇÃO** PODE INTRODUIZIR **FLUXOS BLOQUEANTES, MAIOR DIFICULDADE DE ESCALABILIDADE E ACOPLAMENTO** ENTRE O ORQUESTRADOR E OS SERVIÇOS, ALÉM DE REPRESENTAR UM POSSÍVEL PONTO ÚNICO DE FALHA — AINDA QUE MITIGÁVEL COM REDUNDÂNCIA.

ORQUESTRAÇÃO ASSÍNCRONA



ORQUESTRAÇÃO ASSÍNCRONA - IMPLEMENTAÇÃO

- **Hangfire:** Agendamento de uma rotina *fire-and-forget* para atuar como orquestrador (exige controle manual de estado)
- **Background service:** Execução em segundo plano do orquestrador (exige controle manual de estado)
- **Workflow Core:** Motor para execução de processos de longa duração (estado persistido e controlado pela biblioteca)

ORQUESTRAÇÃO ASSÍNCRONA

```
public class Workflow : IWorkflow<WorkflowData>
{
    public string Id => "OrderProcessingWorkflow";

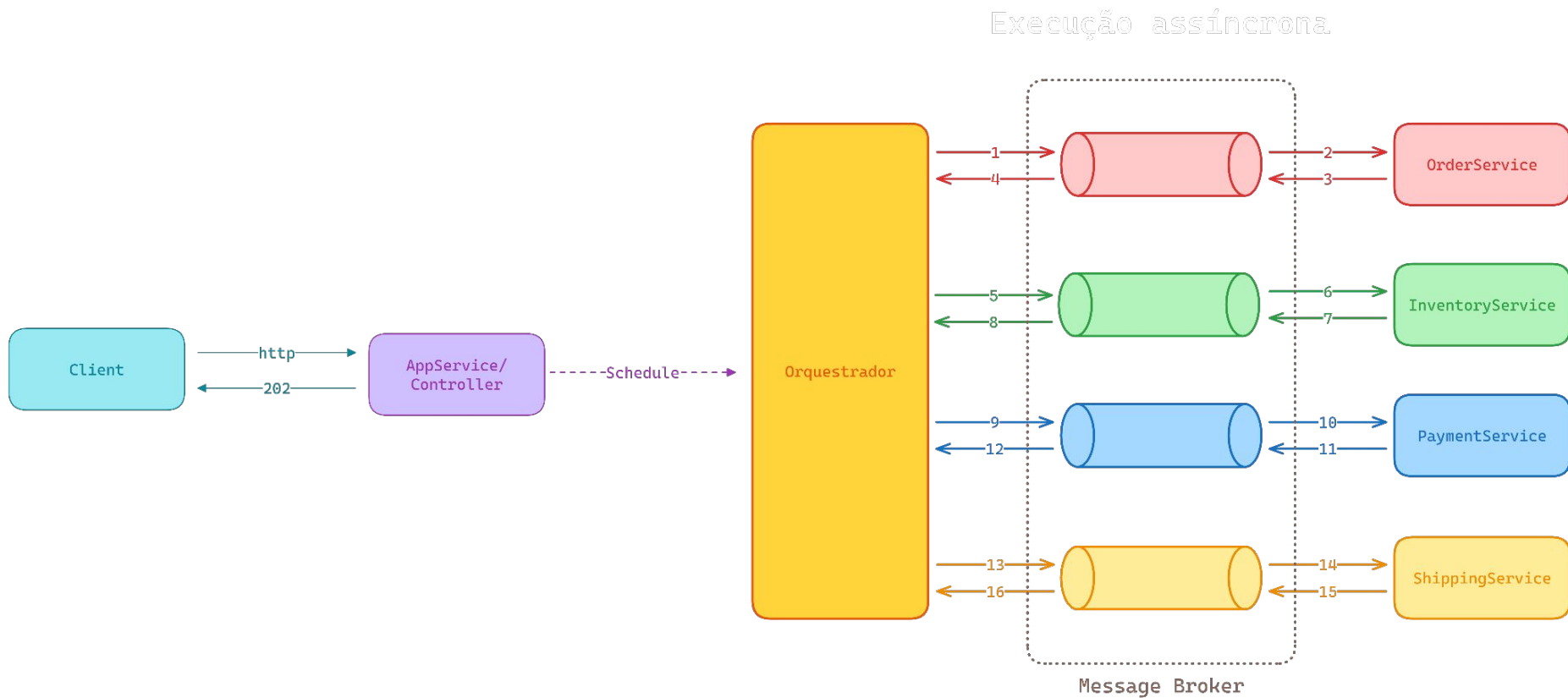
    public int Version => 1;

    public void Build(IWorkflowBuilder<WorkflowData> builder)
    {
        builder
            .StartWith<CreateOrderStep>()
            .Then<CreatePaymentStep>()
            .Then<UpdateInventoryService>()
            .Then<CreateShippingService>();
    }
}
```

WORKFLOW CORE - HANDS ON



ORQUESTRAÇÃO ASSÍNCRONA COM MESSAGE BROKER

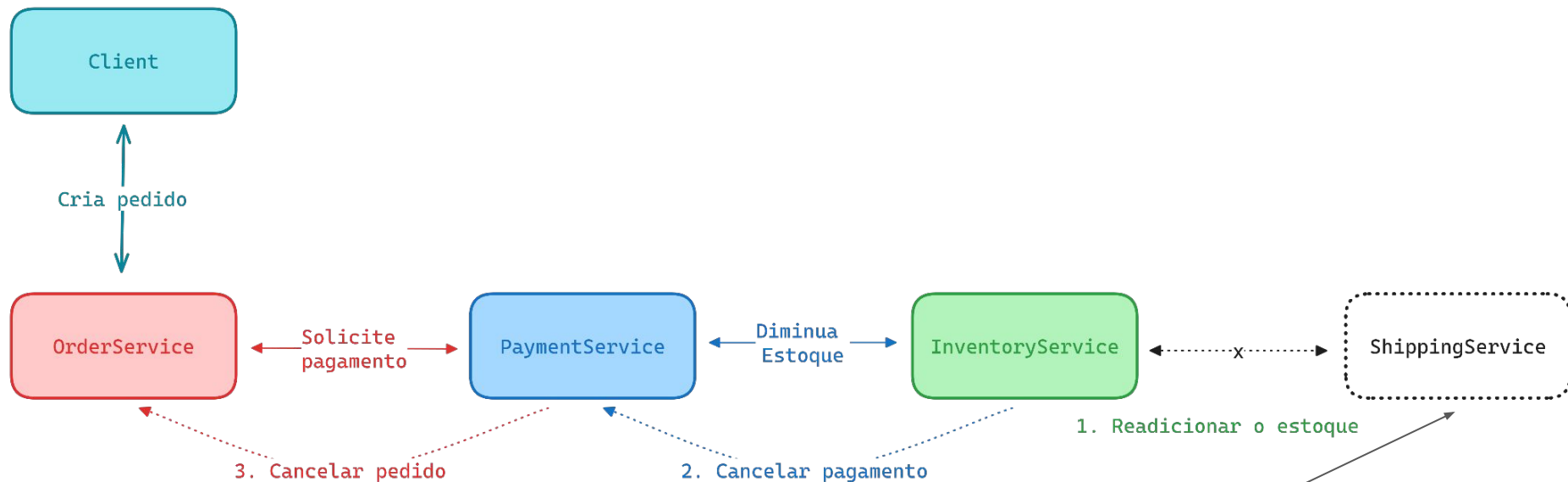


COREOGRAFIA

COREOGRAFIA

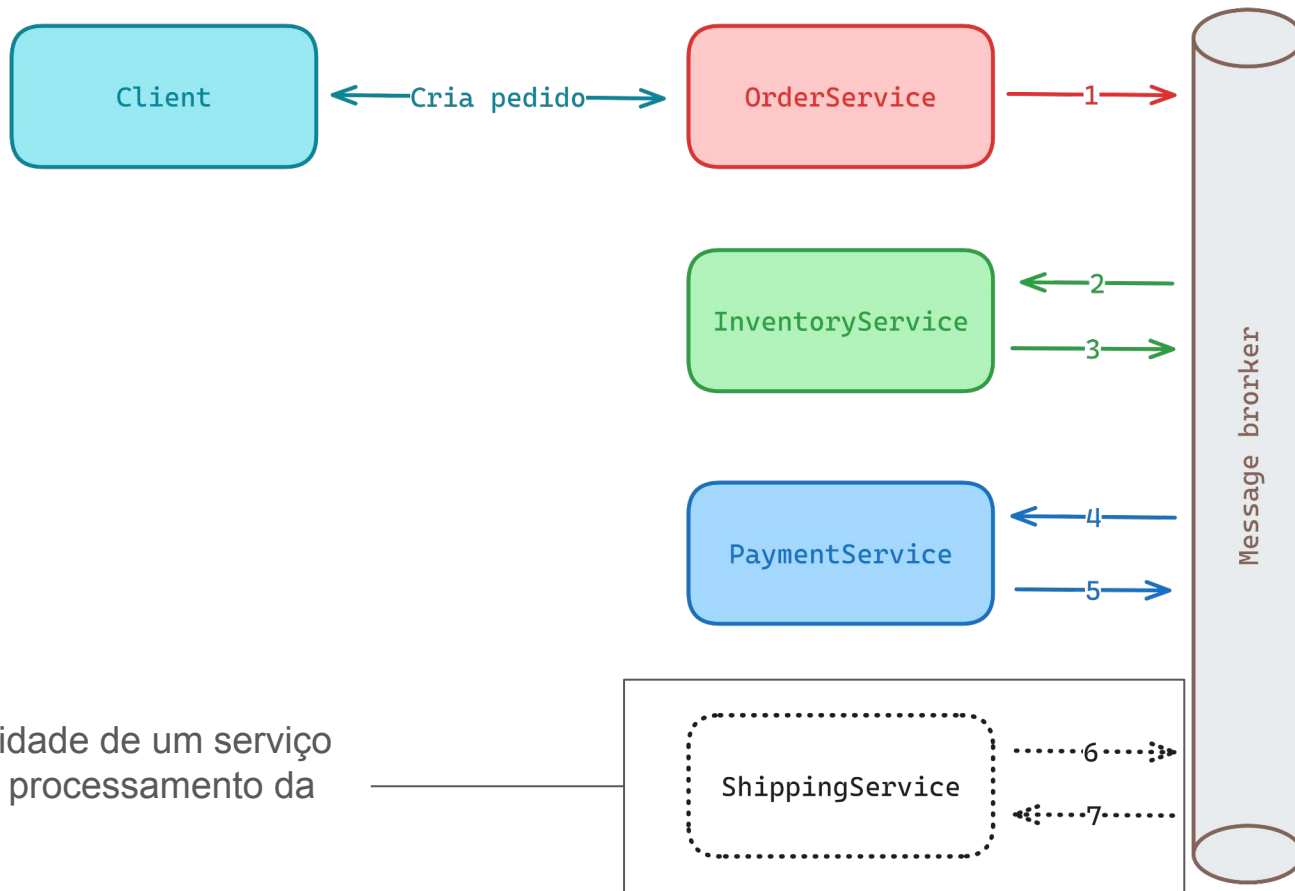
NA **COREOGRAFIA**, OS SERVIÇOS COLABORAM POR MEIO DA TROCA DE MENSAGENS, **SEM UM ORQUESTRADOR CENTRAL**, PERMITINDO A EVOLUÇÃO DO FLUXO COM A ADIÇÃO DE NOVOS SERVIÇOS, MAS EXIGINDO O USO DE **TRANSAÇÕES COMPENSATÓRIAS** PARA TRATAMENTO DE ERROS.

COREOGRAFIA TRATAMENTO DE ERROS



O que fazer caso algum serviço retorne erro ou esteja indisponível?

COREOGRAFIA - BROKER BASED

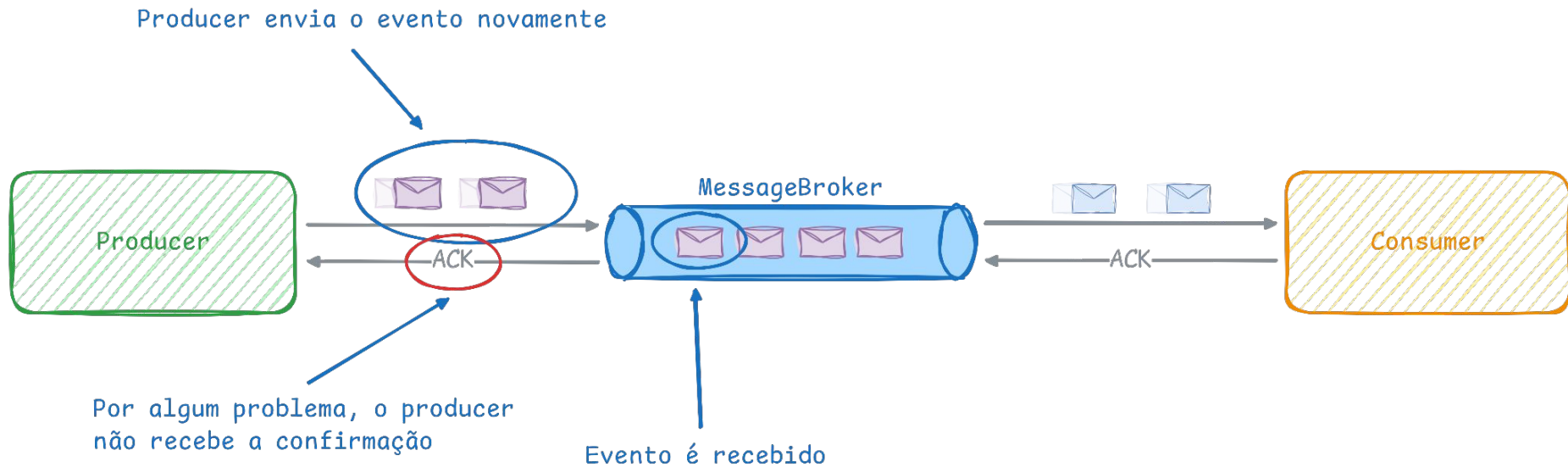


CAPÍTULO 2

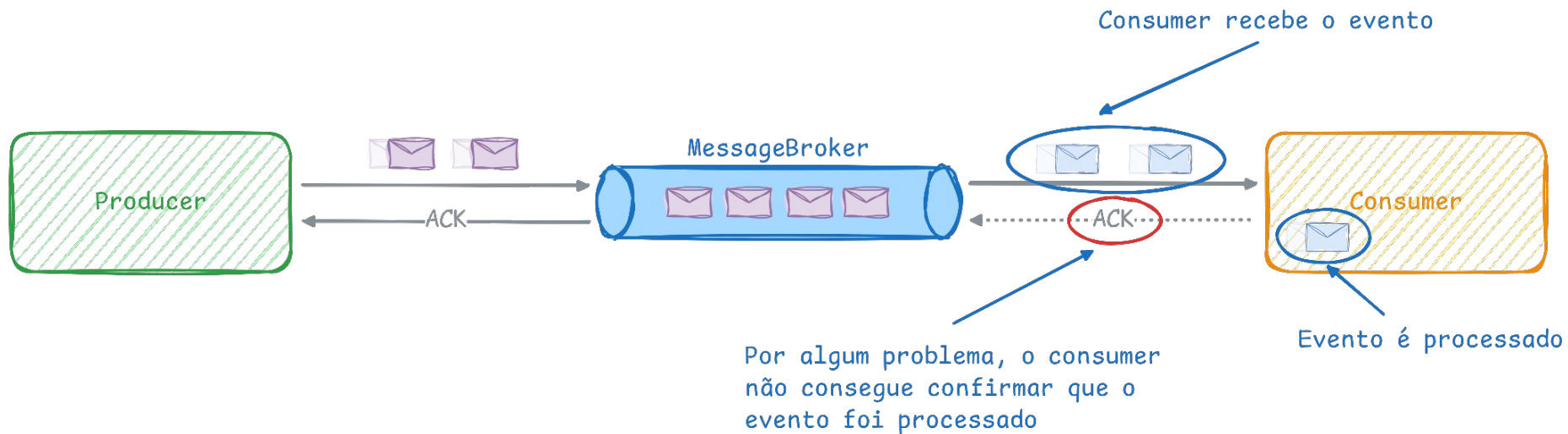
GARANTIA DE ENTREGA



PROBLEMA ACK - PRODUCER



PROBLEMA ACK - CONSUMER



CONFIGURAÇÃO DE GARANTIA DE ENTREGA

AT LEAST ONCE: GARANTE A ENTREGA PELO MENOS UMA VEZ, COM CHANCES DE DUPLICIDADE

AT MOST ONCE: GARANTE A ENTREGA NO MÁXIMO UMA VEZ, COM CHANCES DE NÃO SER ENTREGUE

EXACTLY ONCE: GARANTE A ENTREGA EXATAMENTE UMA VEZ, SEM PERDA DE MENSAGENS OU DUPLICADAS

PROBLEMA ACK - PRODUCER

```
app.MapPost("/order", async (
    OrderDbContext context,
    QueueService queueService,
    ILogger<Program> logger,
    CancellationToken ct) =>
{
    // Order creation logic
    var order = new Order
    {
        Id = Guid.NewGuid().ToString(),
        Status = "Created"
    };

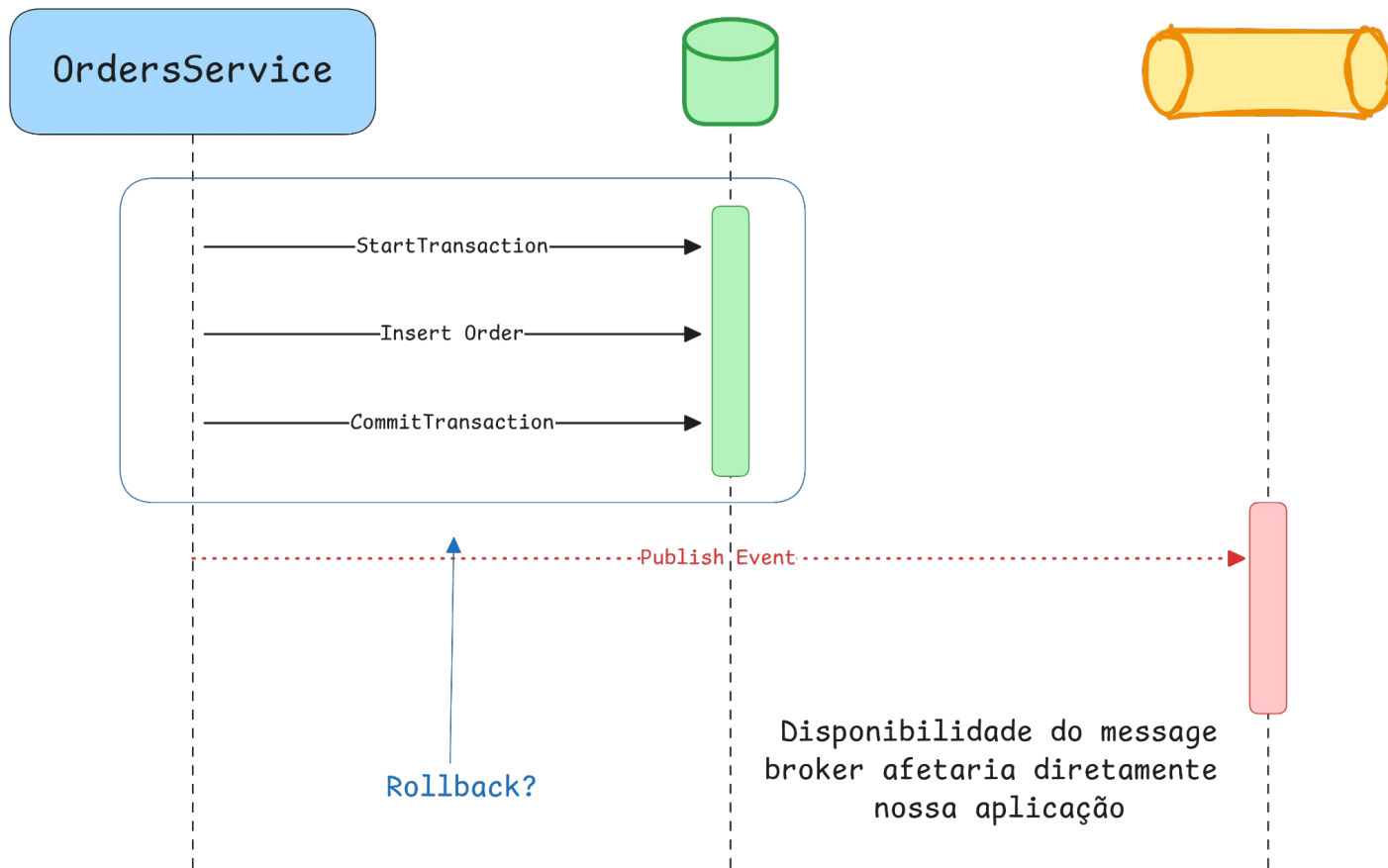
    context.Orders.Add(order);
    await context.SaveChangesAsync(ct);

    await queueService.PublishAsync(QueueNames.OrderCreatedQueue, new OrderCreatedEvent(order.Id));

    logger.LogInformation("Order {OrderId} created", order.Id);
    return Results.Ok($"Order {order.Id} created");
})
.WithName("CreateOrder");
```

Tenho uma operação “atômica”?

ENTENDENDO O PROBLEMA - ATOMICIDADE



ENTENDENDO O PROBLEMA - ATOMICIDADE

```
app.MapGet("/order", async (
    OrderDbContext context,
    QueueService queueService,
    ILogger<Program> logger,
    CancellationToken ct) =>
{
    // Order creation logic
    var order = new Order
    {
        Id = Guid.NewGuid().ToString(),
        Status = "Created"
    };

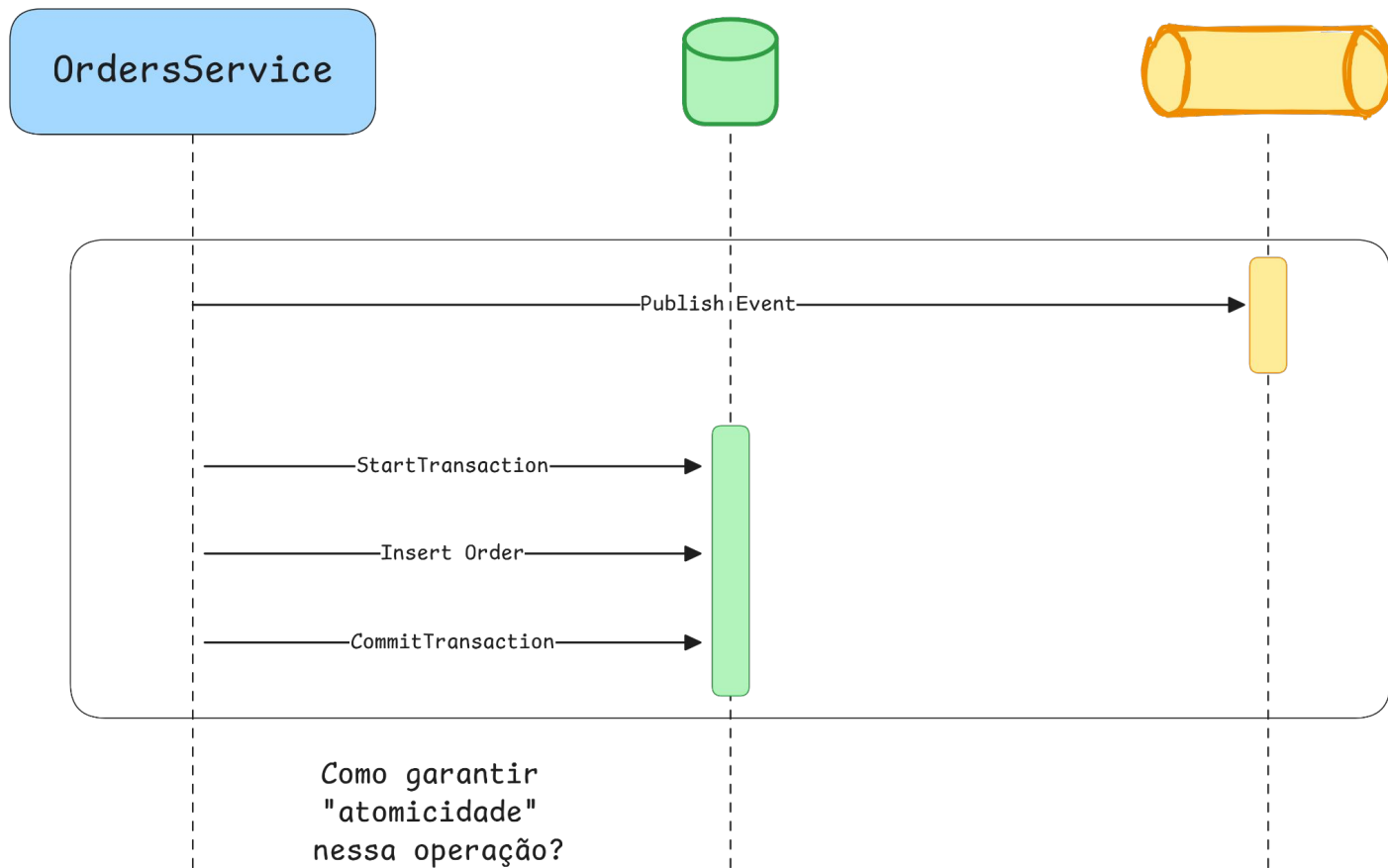
    context.Orders.Add(order);

    await queueService.PublishAsync(QueueNames.OrderCreatedQueue, new OrderCreatedEvent(order.Id));

    await context.SaveChangesAsync(ct);

    logger.LogInformation("Order {OrderId} created", order.Id);
    return Results.Ok($"Order {order.Id} created");
})
.WithName("CreateOrder");
```

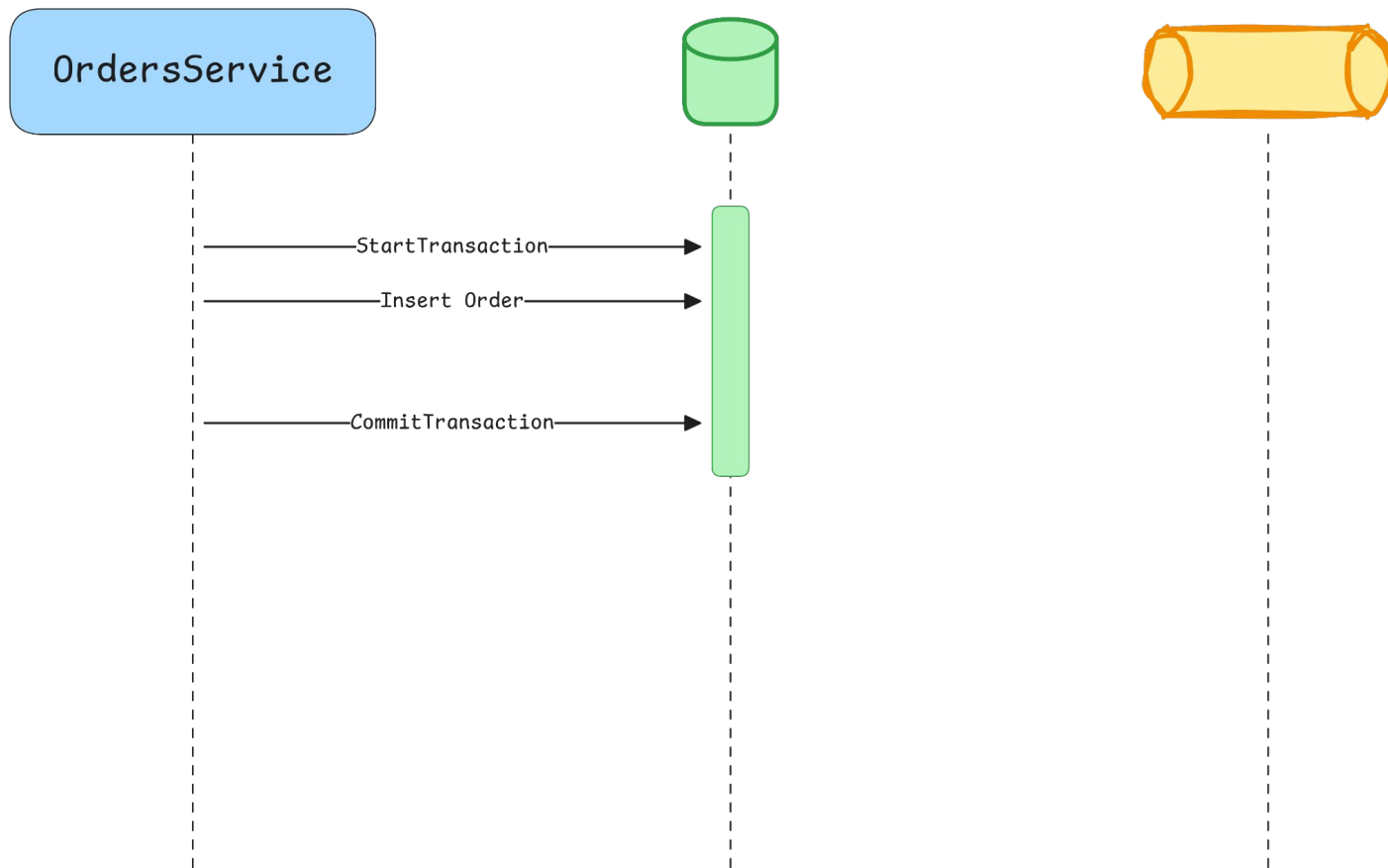
ENTENDENDO O PROBLEMA - ATOMICIDADE



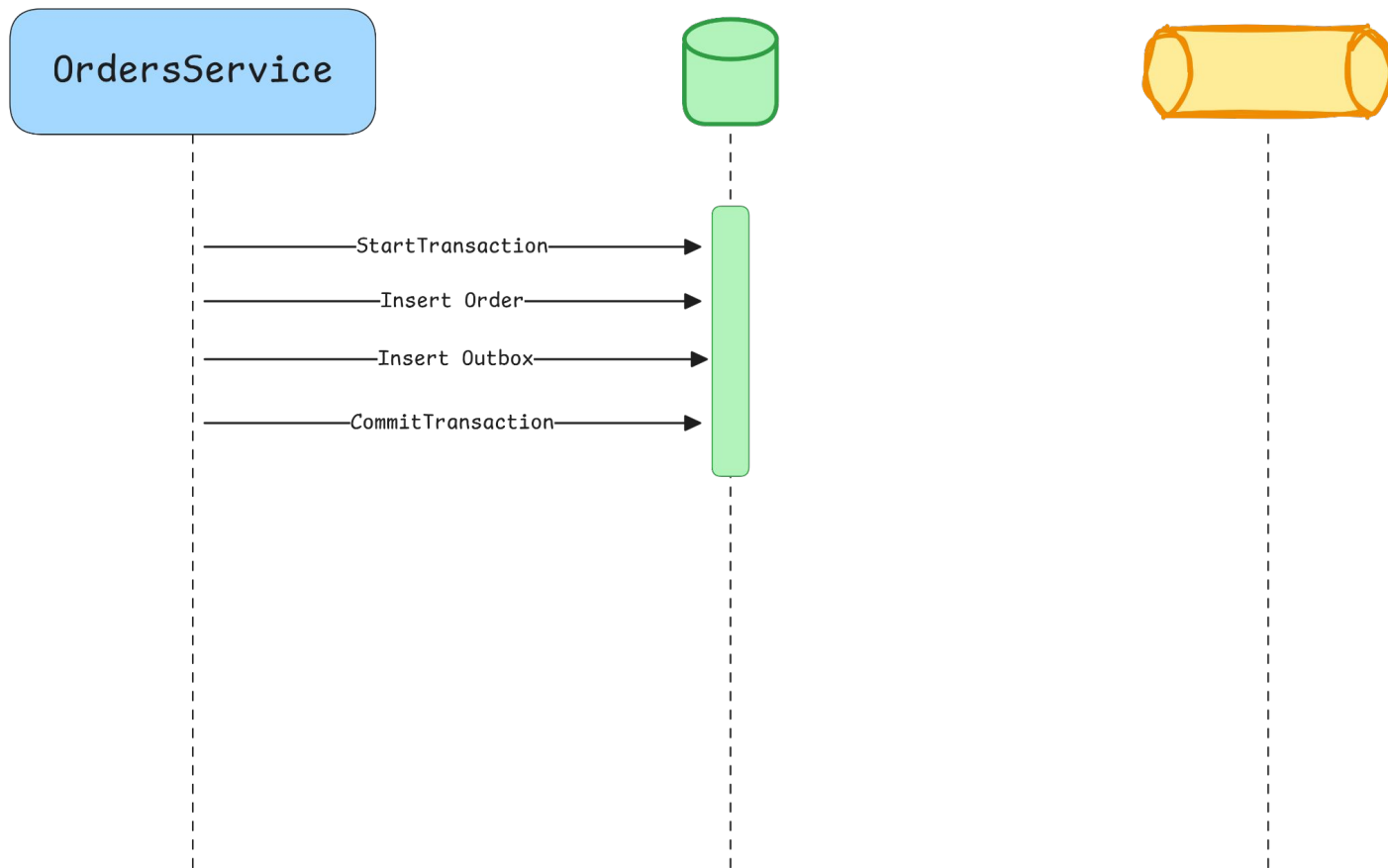
OUTBOX PATTERN

- Garante a entrega da mensagem
- Eventos não são enviados no momento da transação de negócio
- Persiste eventos a serem transmitidos na mesma transação de negócio (ACID)
- Worker/relay envia os eventos em segundo plano

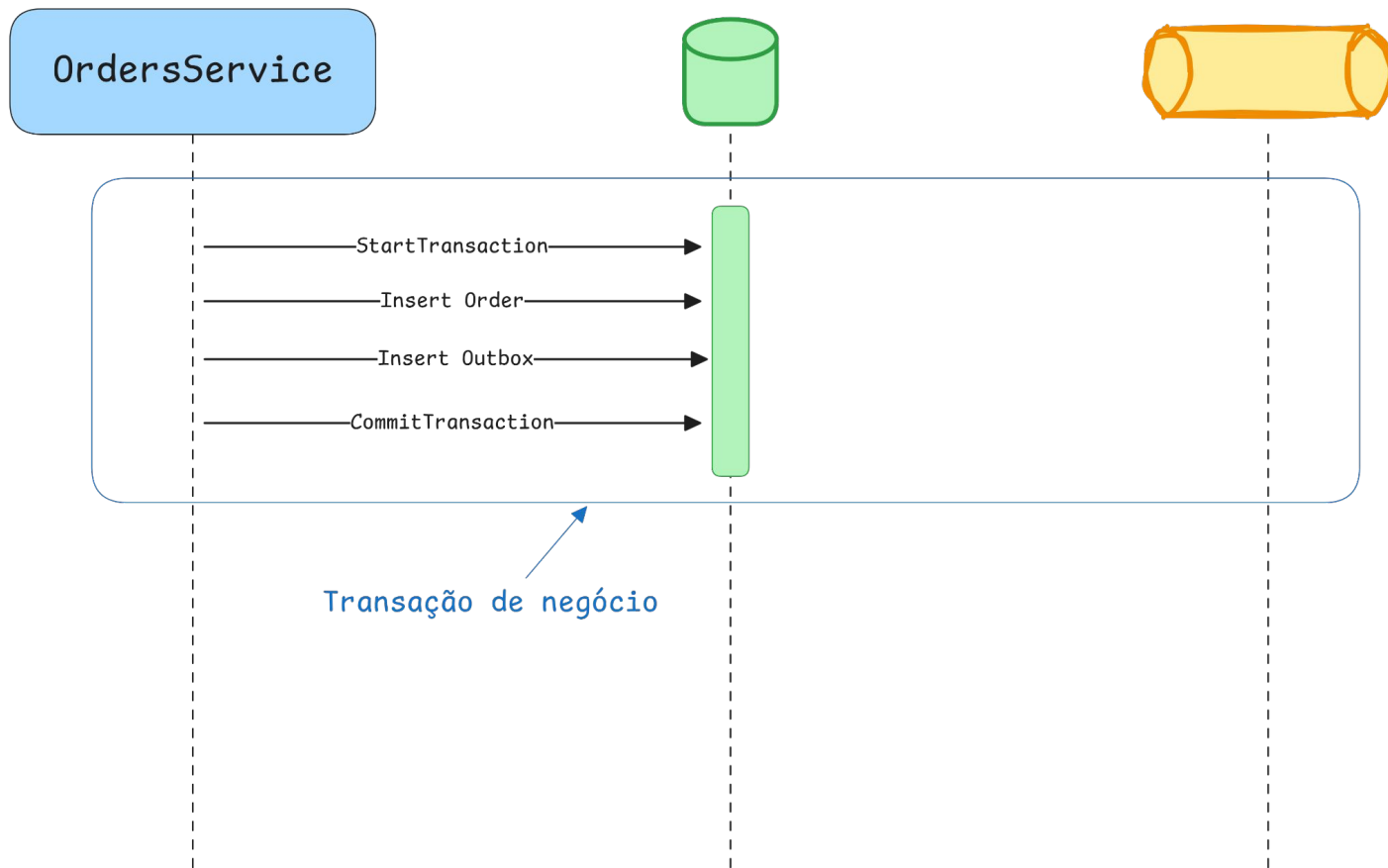
OUTBOX PATTERN



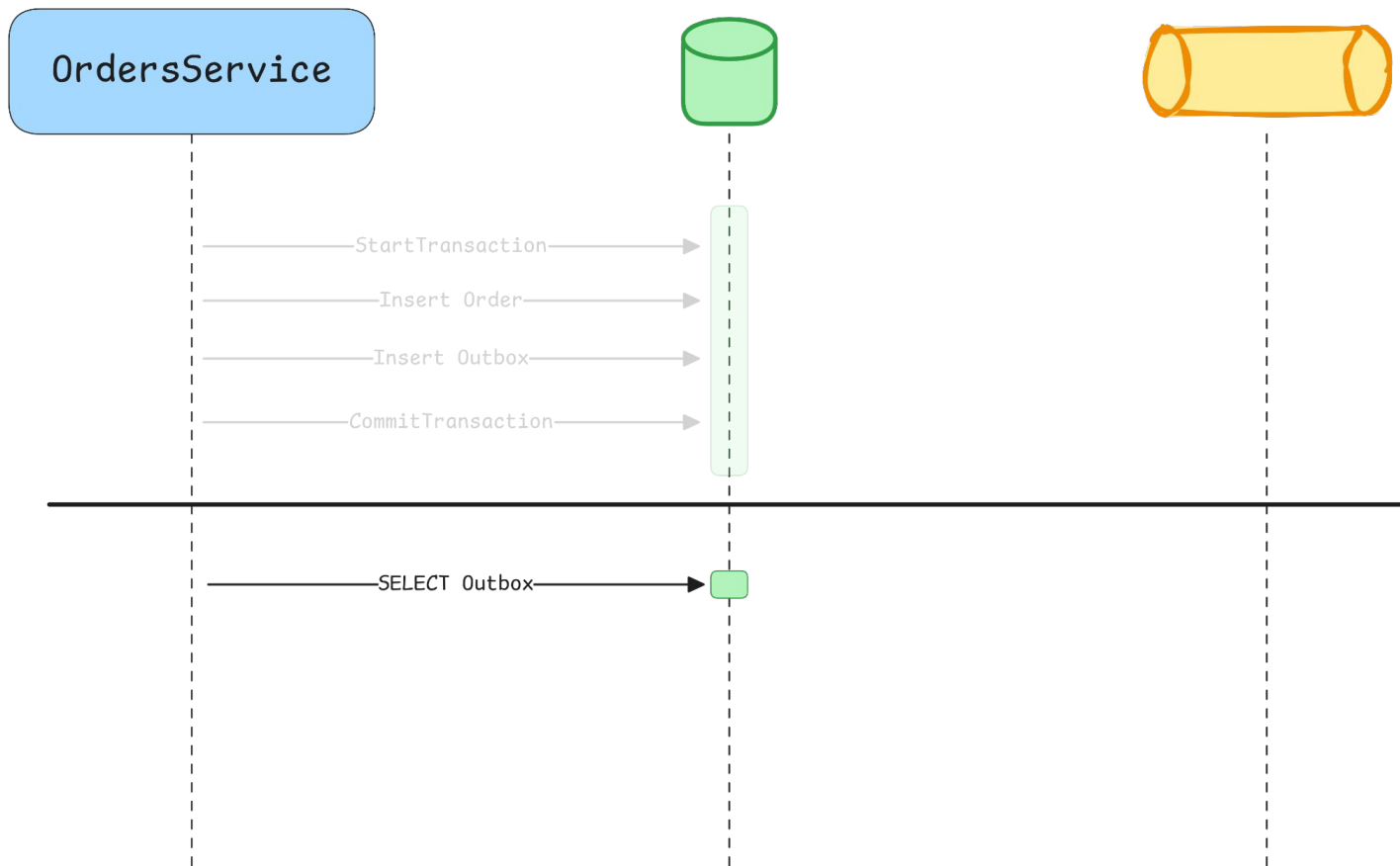
OUTBOX PATTERN



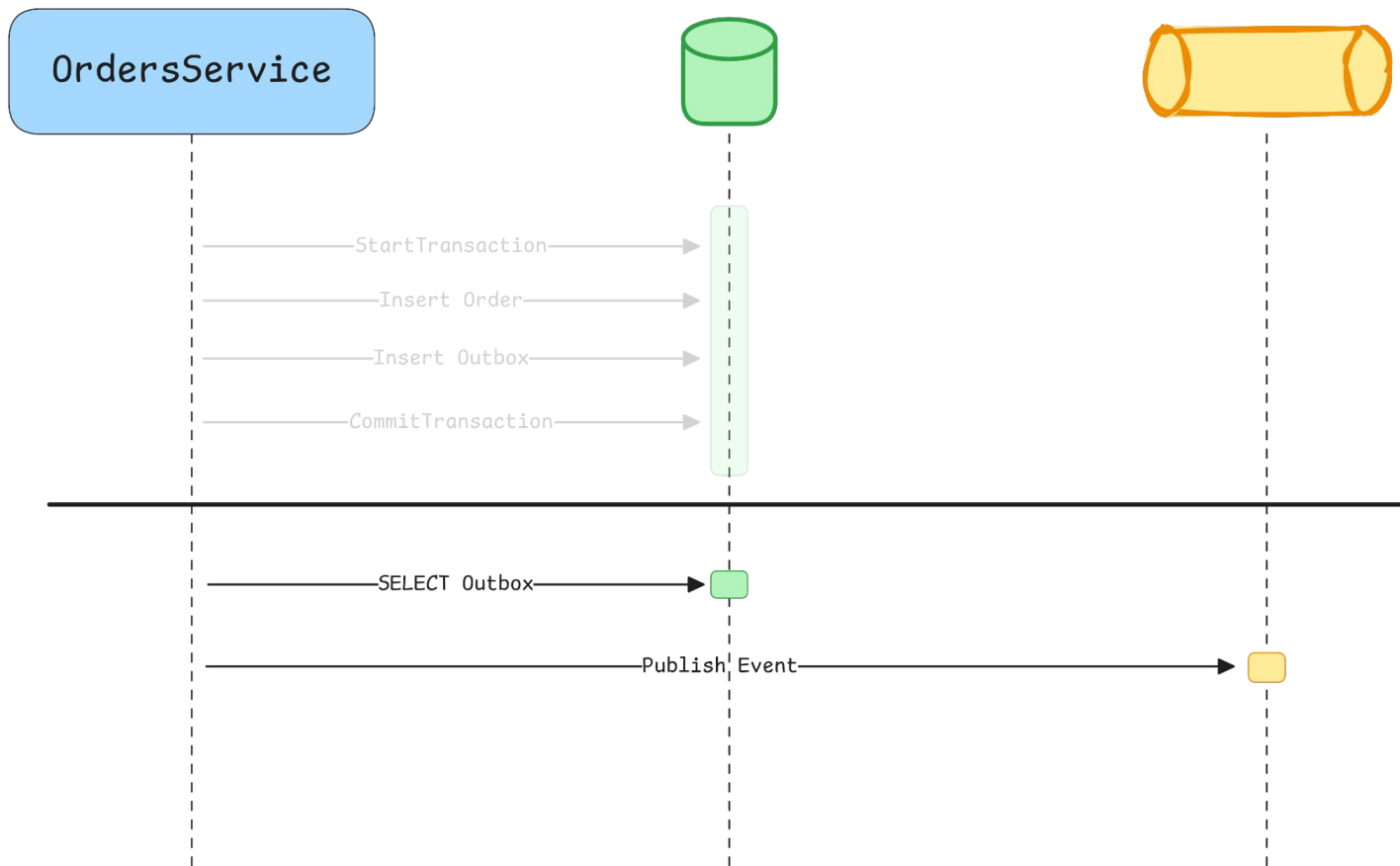
OUTBOX PATTERN



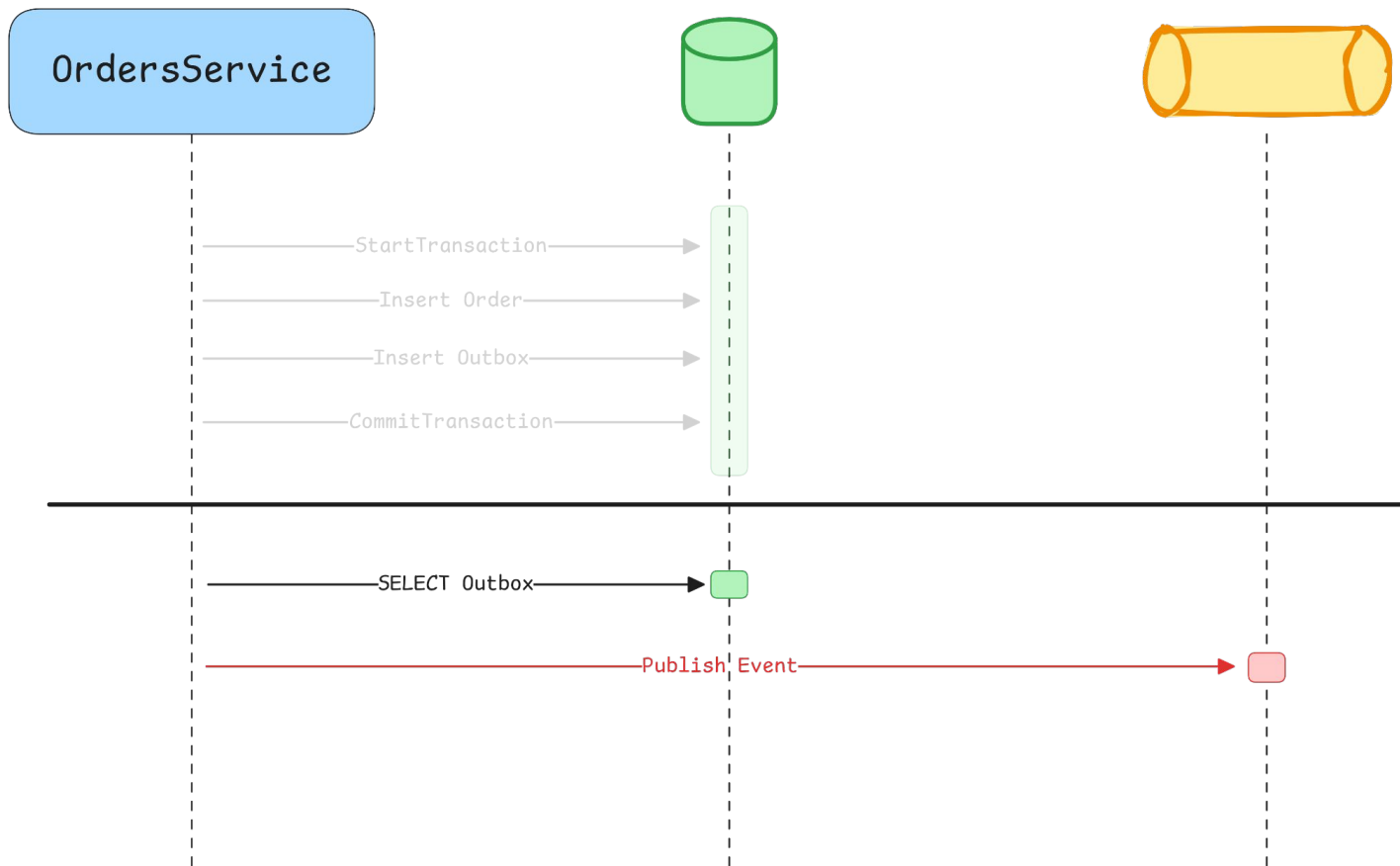
OUTBOX PATTERN



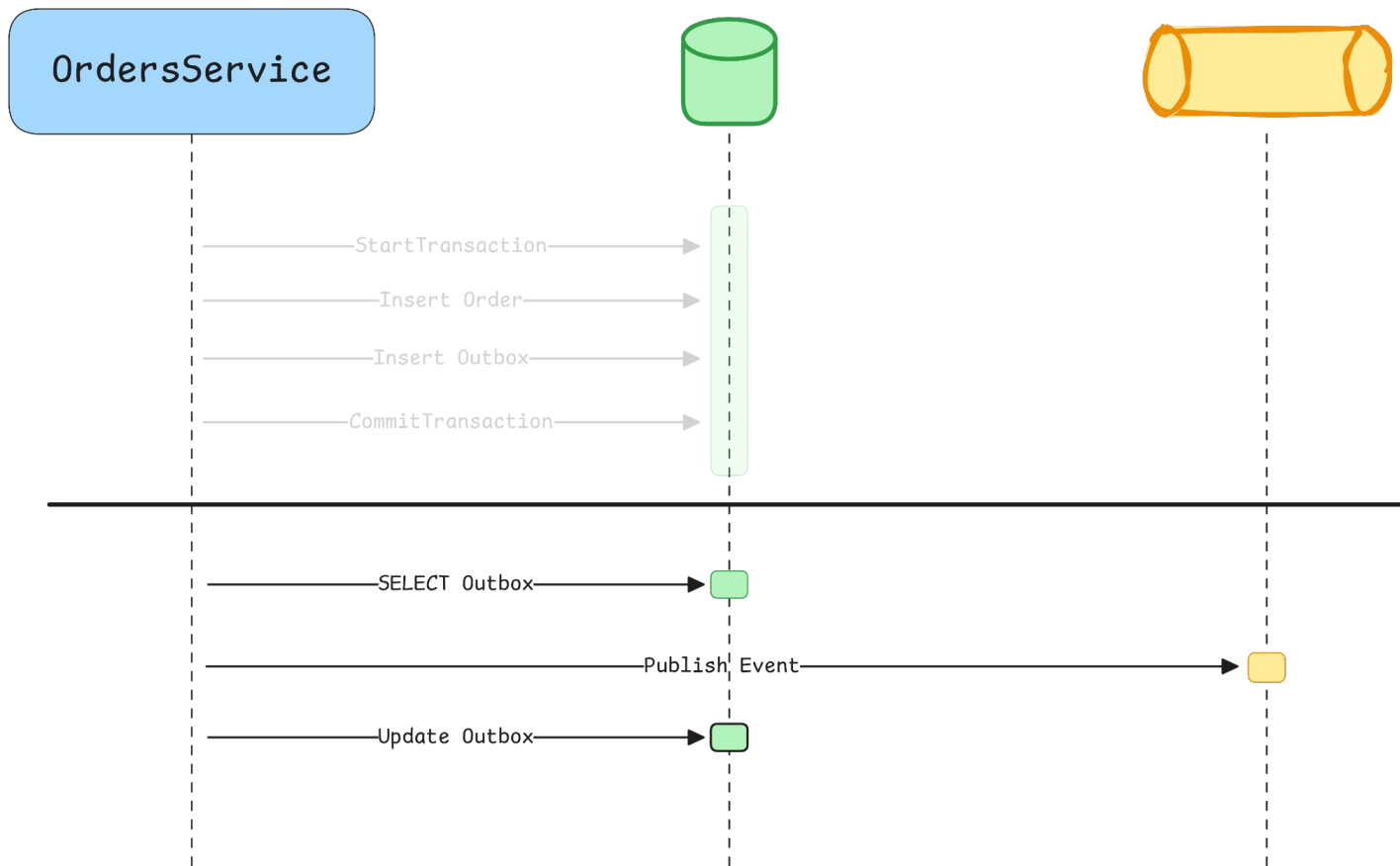
OUTBOX PATTERN



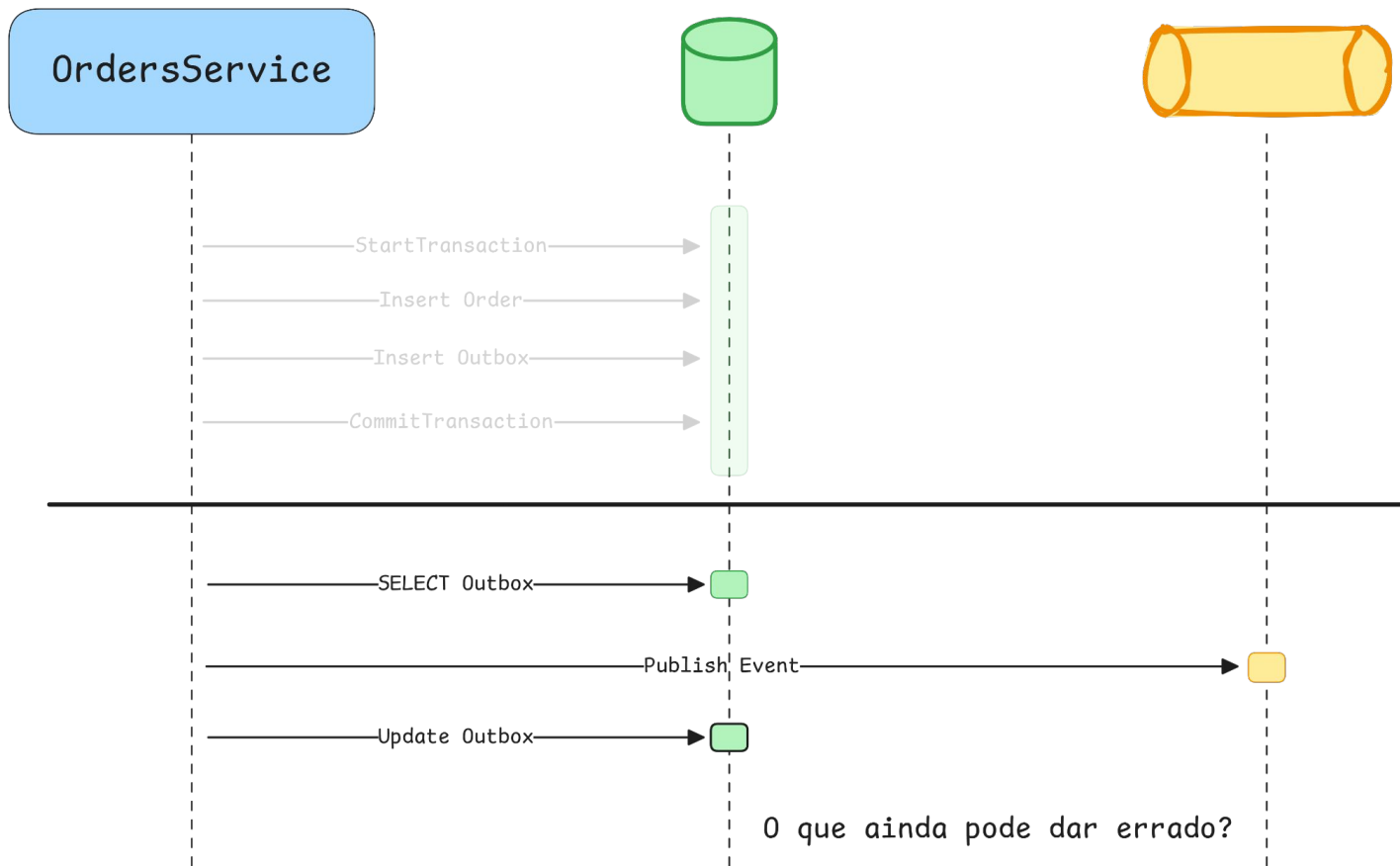
OUTBOX PATTERN



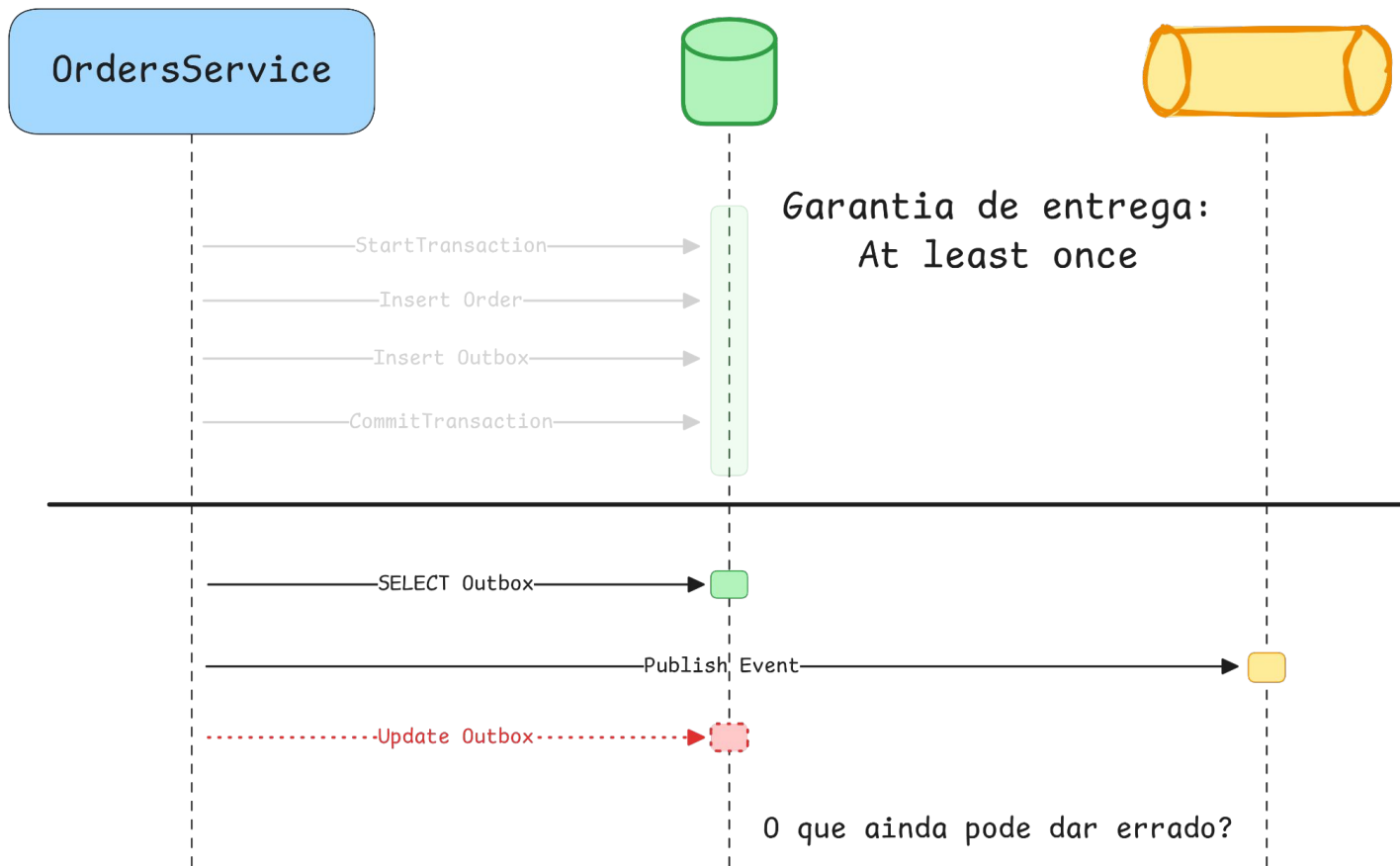
OUTBOX PATTERN



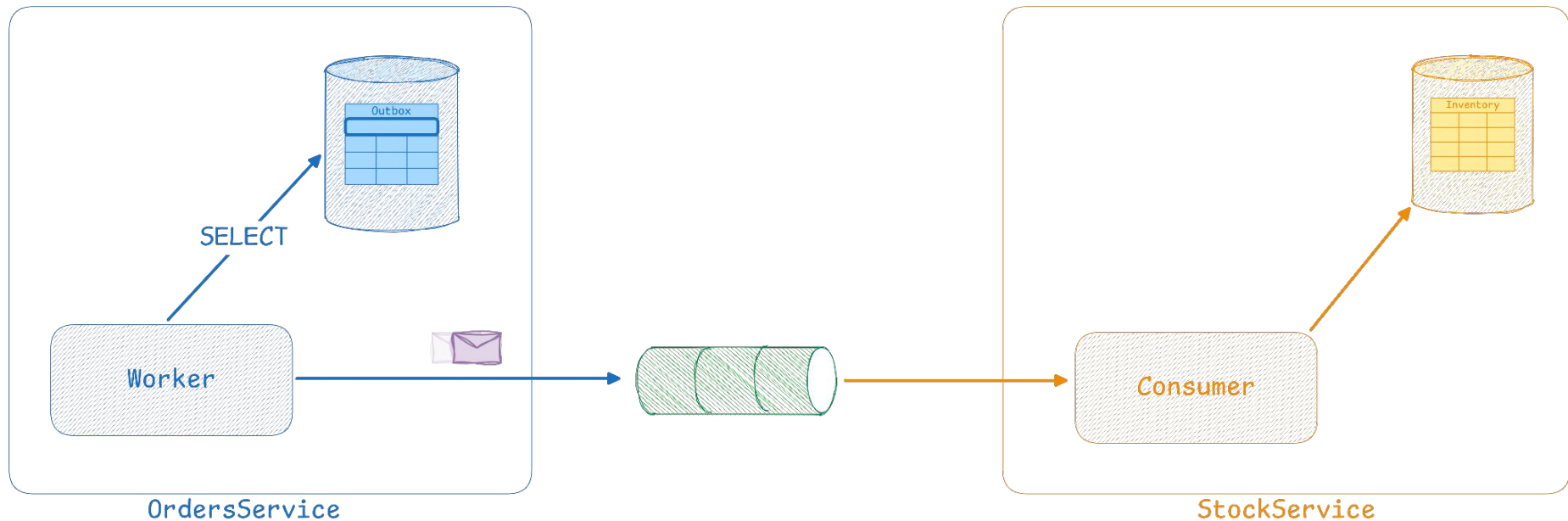
OUTBOX PATTERN



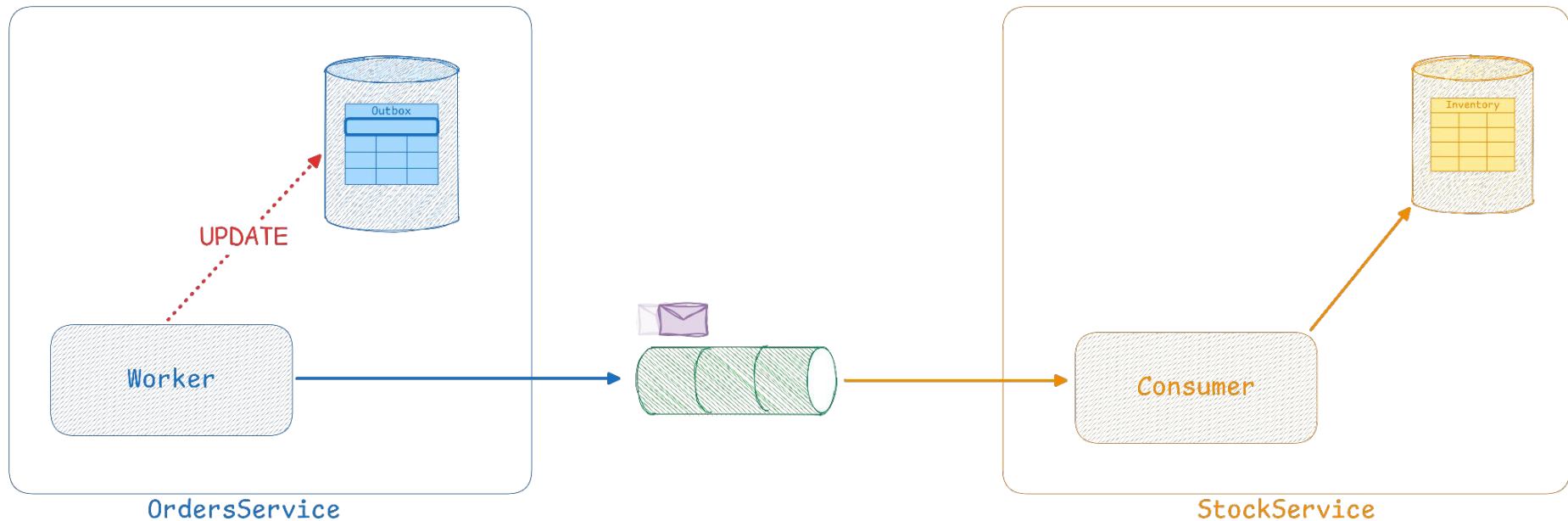
OUTBOX PATTERN



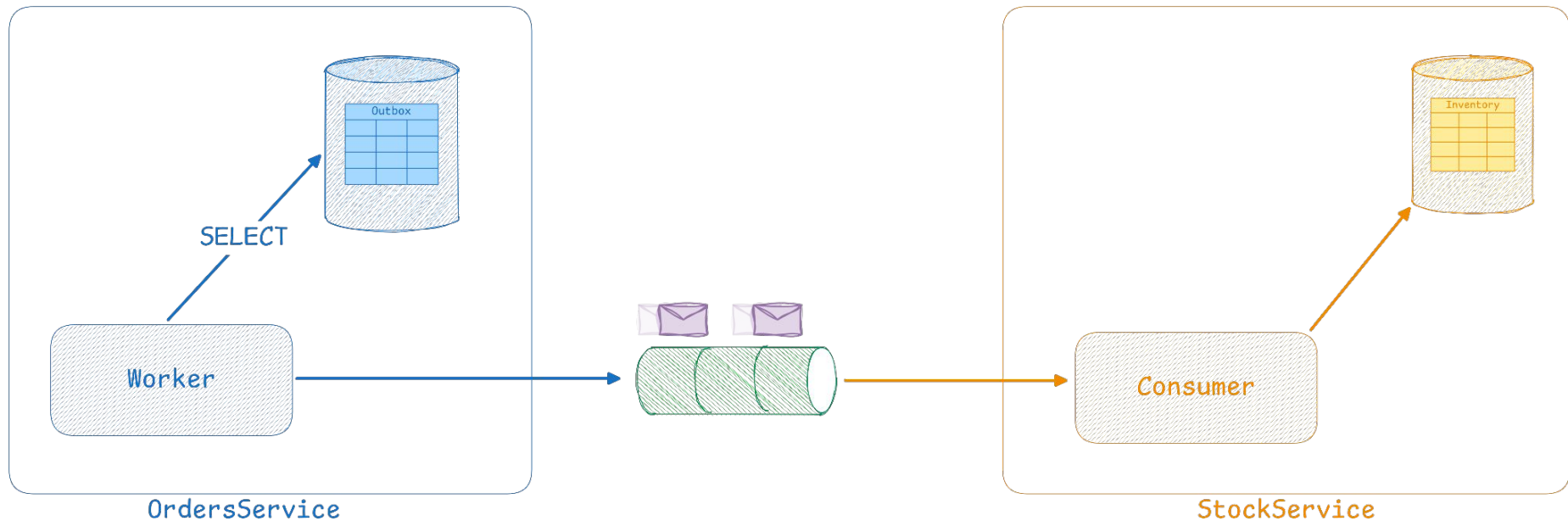
OUTBOX PATTERN - PROBLEMA



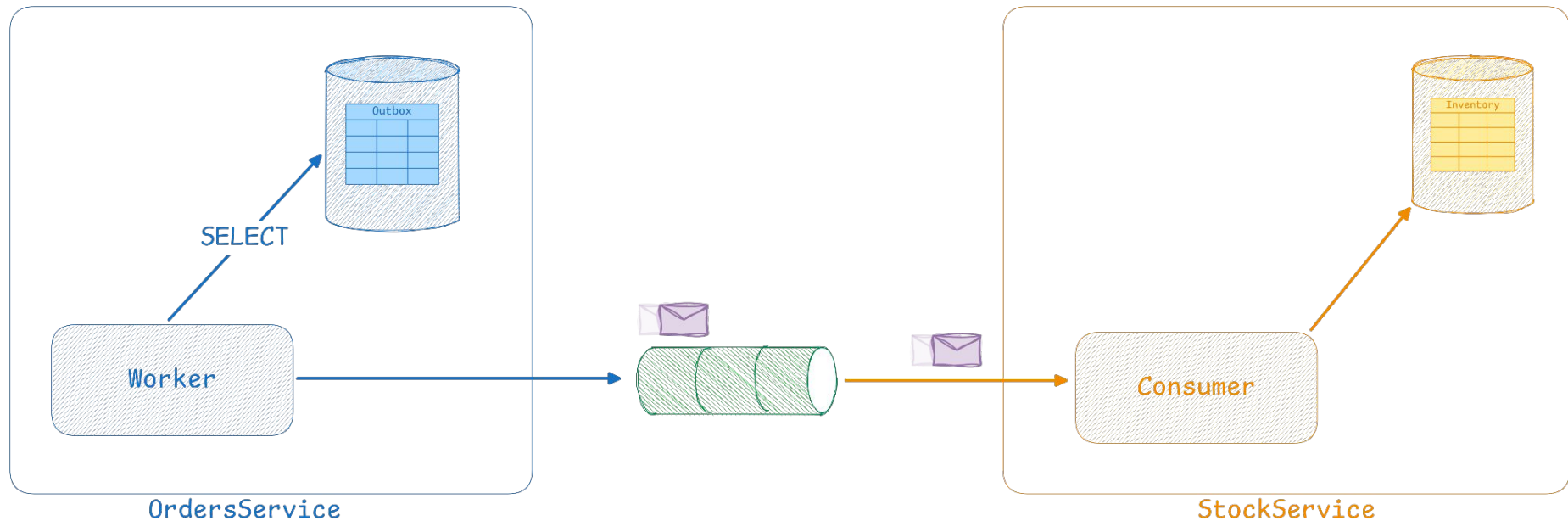
OUTBOX PATTERN - PROBLEMA



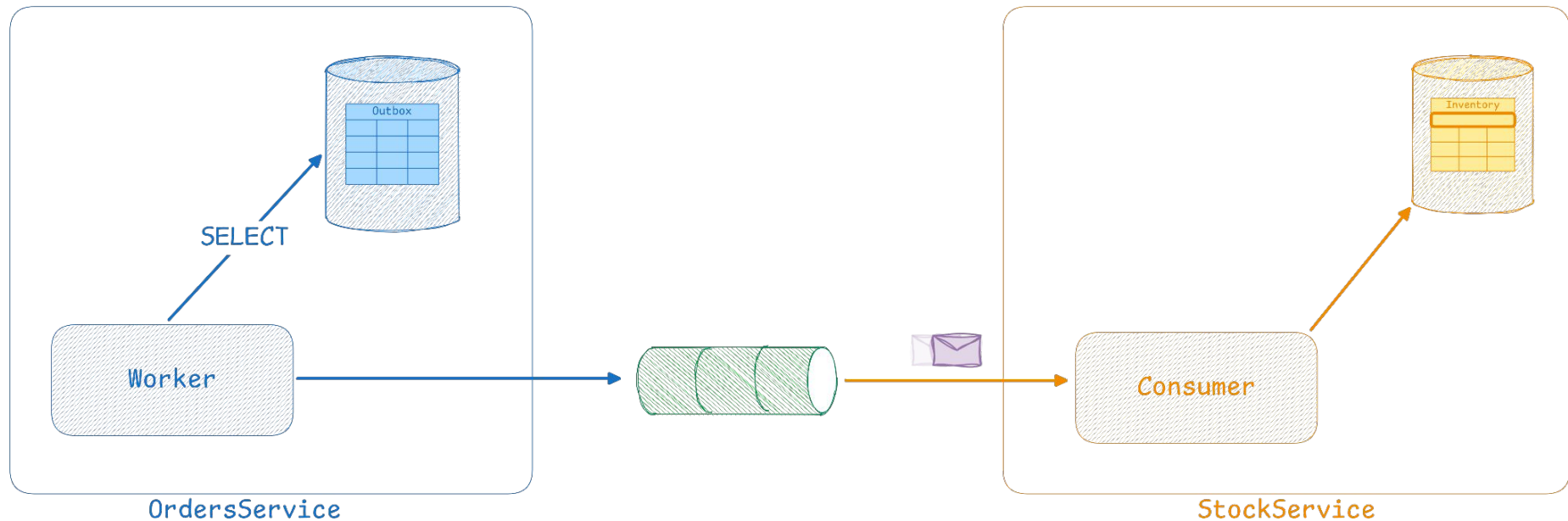
OUTBOX PATTERN - PROBLEMA



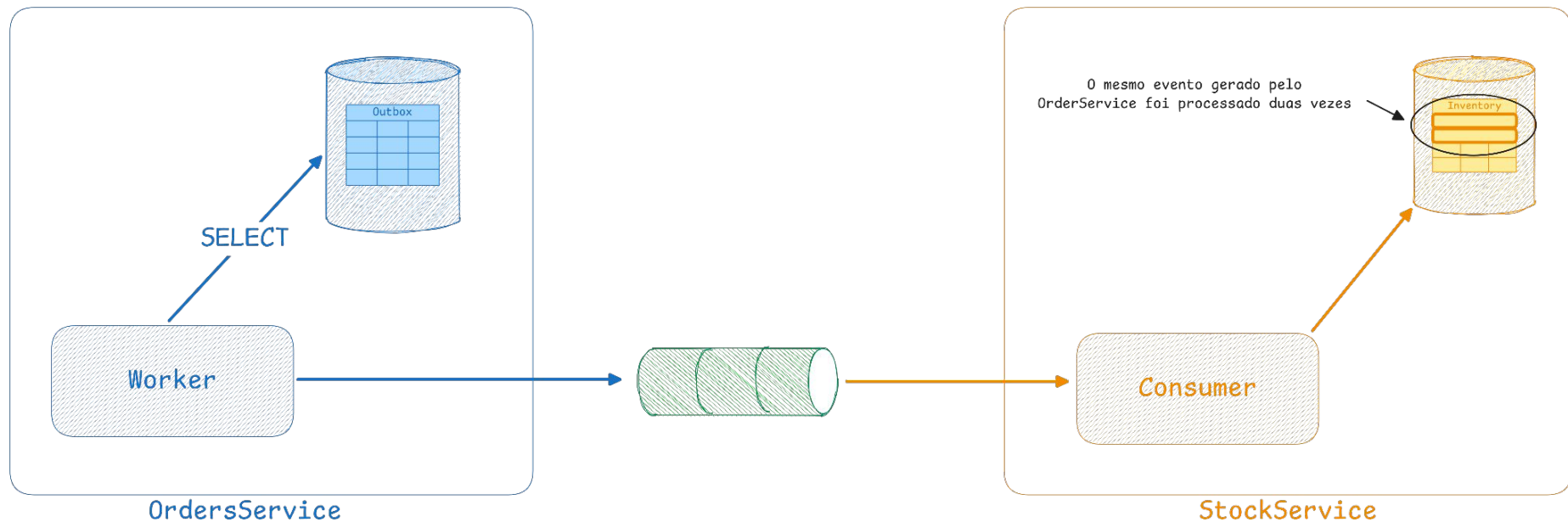
OUTBOX PATTERN - PROBLEMA



OUTBOX PATTERN - PROBLEMA



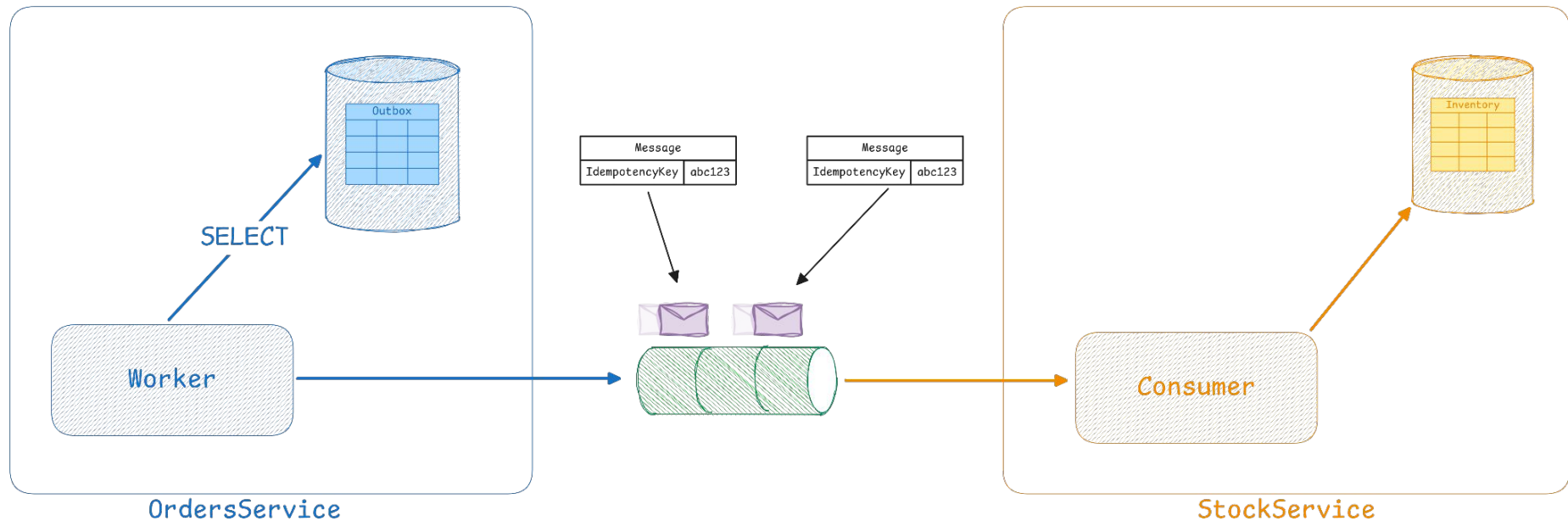
OUTBOX PATTERN - PROBLEMA



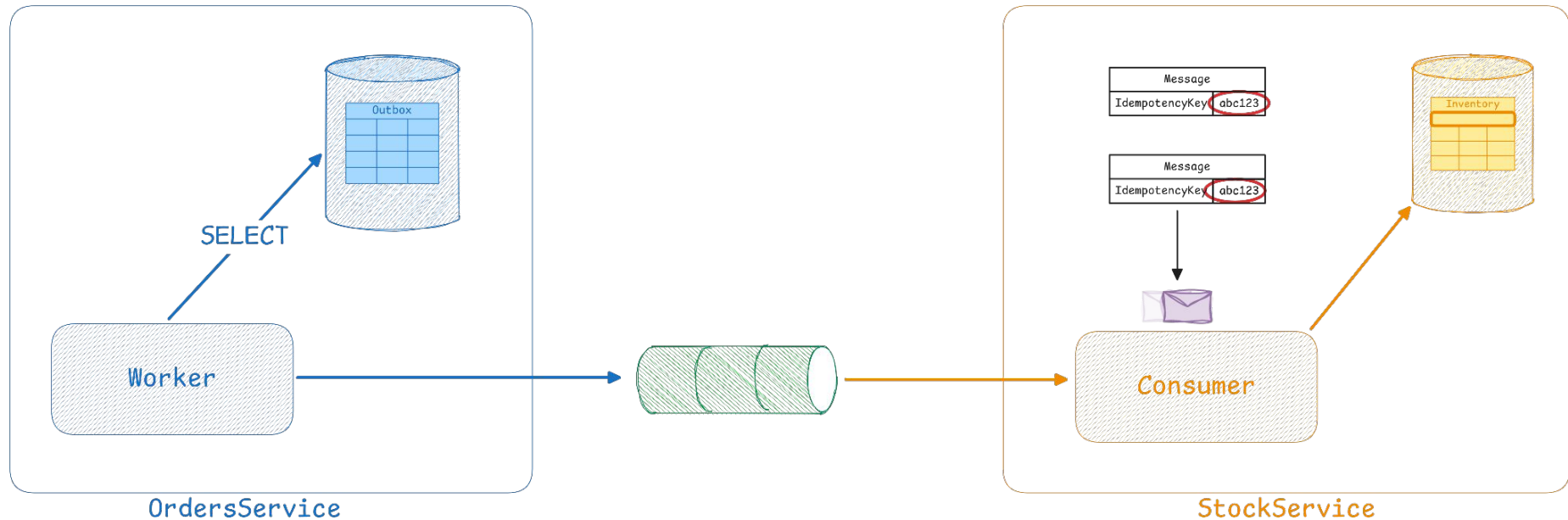
IDEMPOTÊNCIA

IDEMPOTÊNCIA GARANTE QUE UMA OPERAÇÃO PRODUZA SEMPRE O MESMO RESULTADO, MESMO QUANDO EXECUTADA VÁRIAS VEZES, SENDO ESSENCIAL PARA MANTER A CONSISTÊNCIA E PERMITIR **RETRIES** SEGUROS EM **SISTEMAS DISTRIBUÍDOS**.

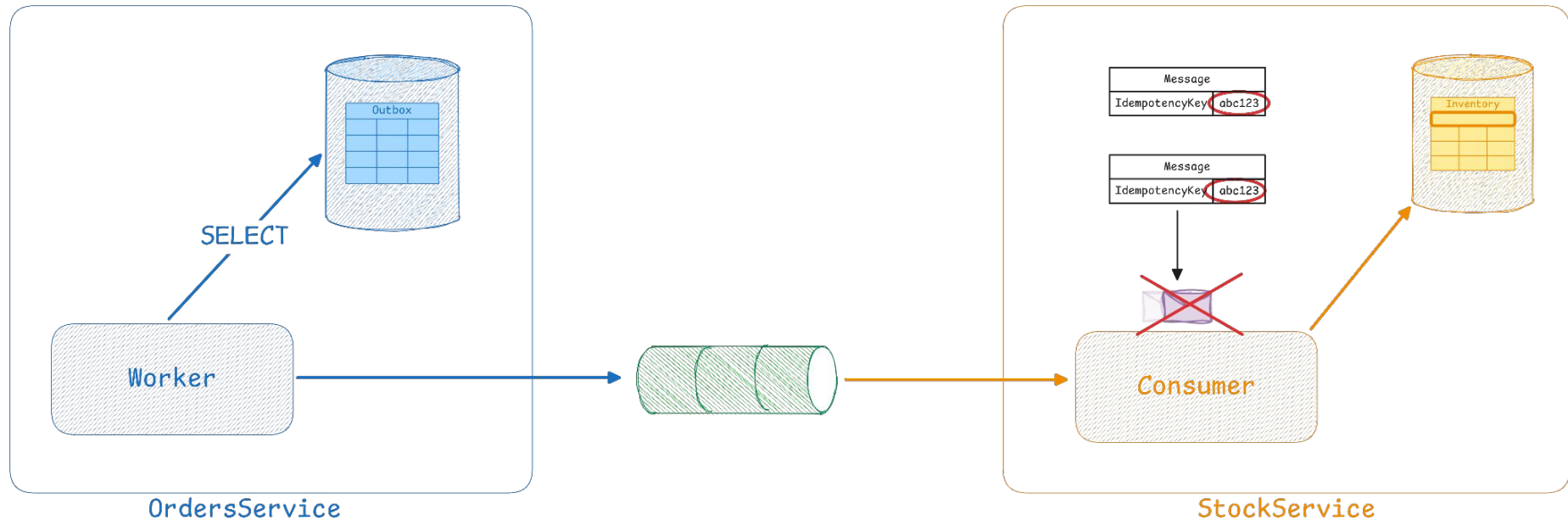
IDEMPOTÊNCIA



IDEMPOTÊNCIA



IDEMPOTÊNCIA



CAPÍTULO 3

BROKERS DE MENSAGERIA





RABBITMQ

- Suporta persistência de mensagens, acks e confirmação de publicação
- Suporta configurações complexas para roteamento de mensagens
- Alta disponibilidade e tolerância a falhas são alcançadas através de clustering, permitindo que vários "nós" trabalhem em conjunto.
- Suporta tanto o modelo **queue** quanto **publish-subscribe**

RABBITMQ - COMPONENTES

Producer

Exchange

Queue

Consumer

RABBITMQ - TIPOS DE EXCHANGE

Direct

REALIZA O ROTEAMENTO DE MENSAGENS PARA FILAS QUE SATISFAZEM EXATAMENTE A ROUTING KEY

Fanout

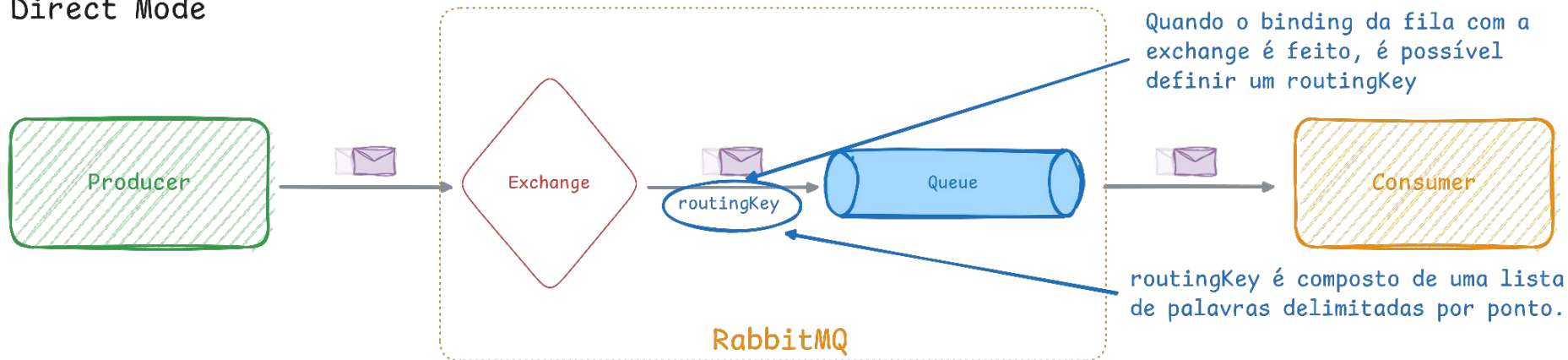
NÃO UTILIZA ROUTING KEY, OU SEJA, ENVIA PARA TODAS AS FILAS “PLUGADAS” NA EXCHANGE

Topic

UTILIZA ROUTING KEYS PARA ENCAMINHAR MENSAGENS, PODENDO UTILIZAR CORINGAS PARA AMPLIAR O ESCOPO DE MENSAGENS CAPTURADAS

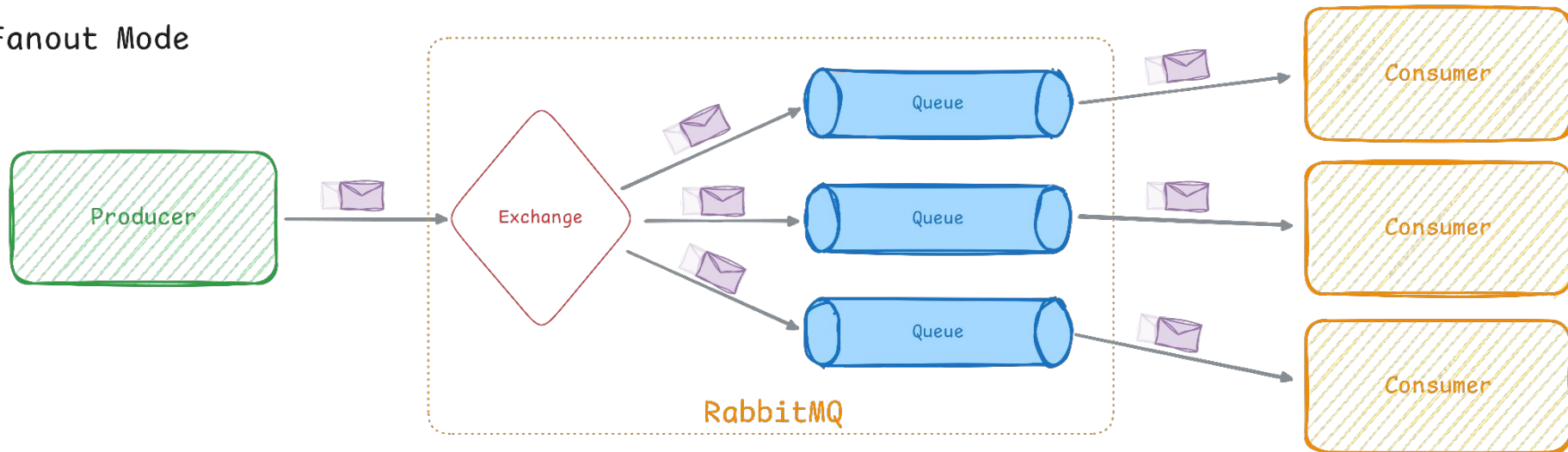
RABBITMQ - DIRECT MODE

Direct Mode



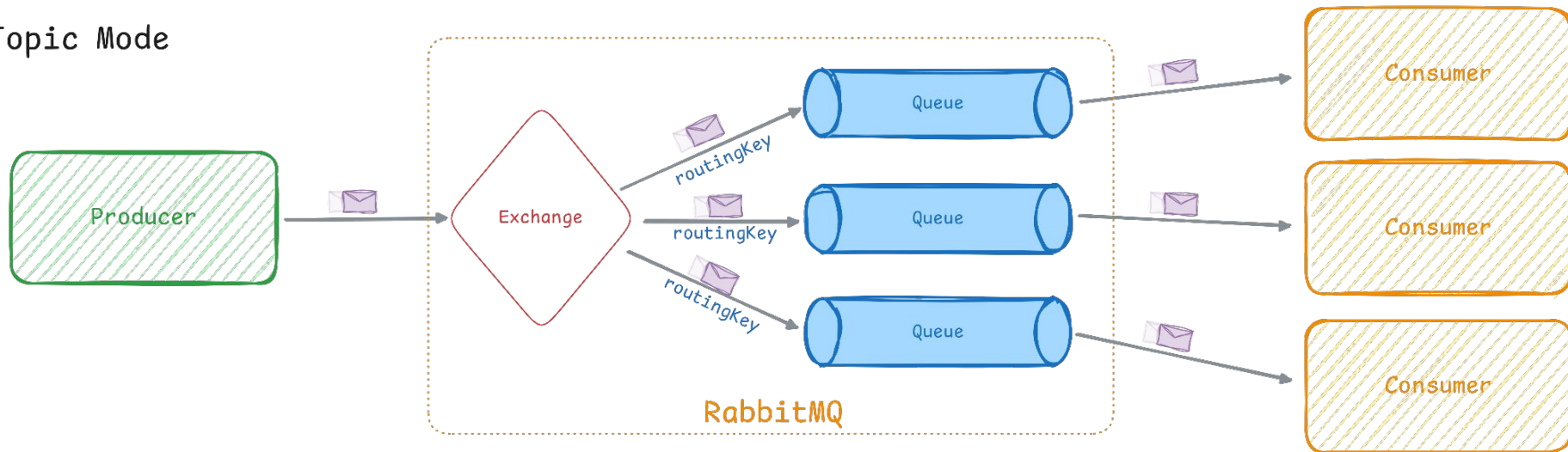
RABBITMQ - FANOUT MODE

Fanout Mode



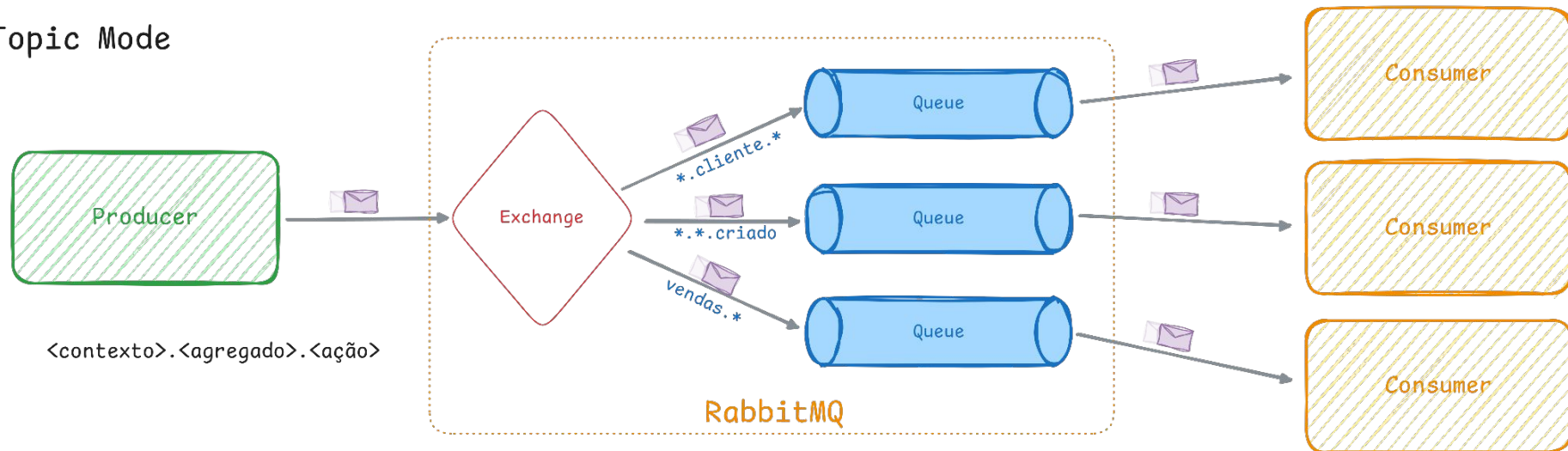
RABBITMQ - TOPIC MODE

Topic Mode



RABBITMQ - PADRÃO DE NOMENCLATURA

Topic Mode





Azure Service Bus

AZURE SERVICE BUS

- Gestão em nuvem (Azure)
- Necessidade mínima de configuração manual
- Escalabilidade é feita de forma automática
- Suporta tanto o modelo **queue** quanto **publish-subscribe**

AZURE SERVICE BUS - COMPONENTES

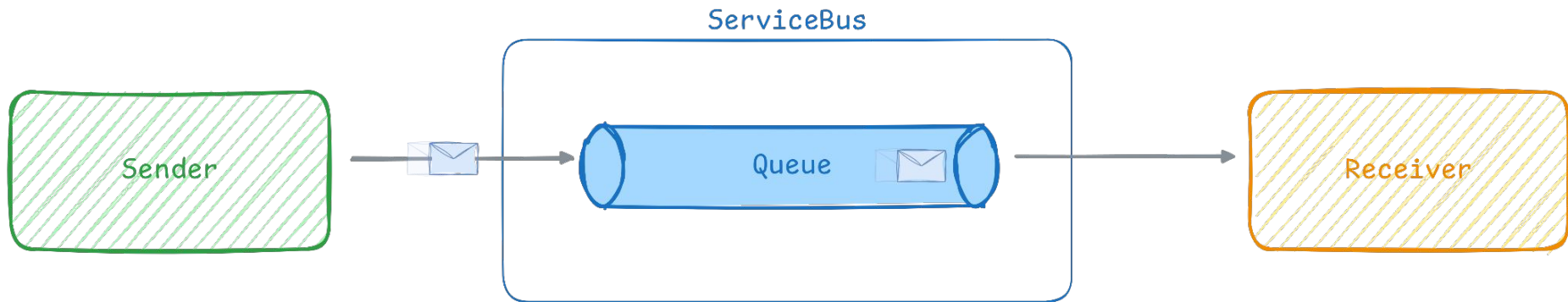
Queue

Topic

Subscription

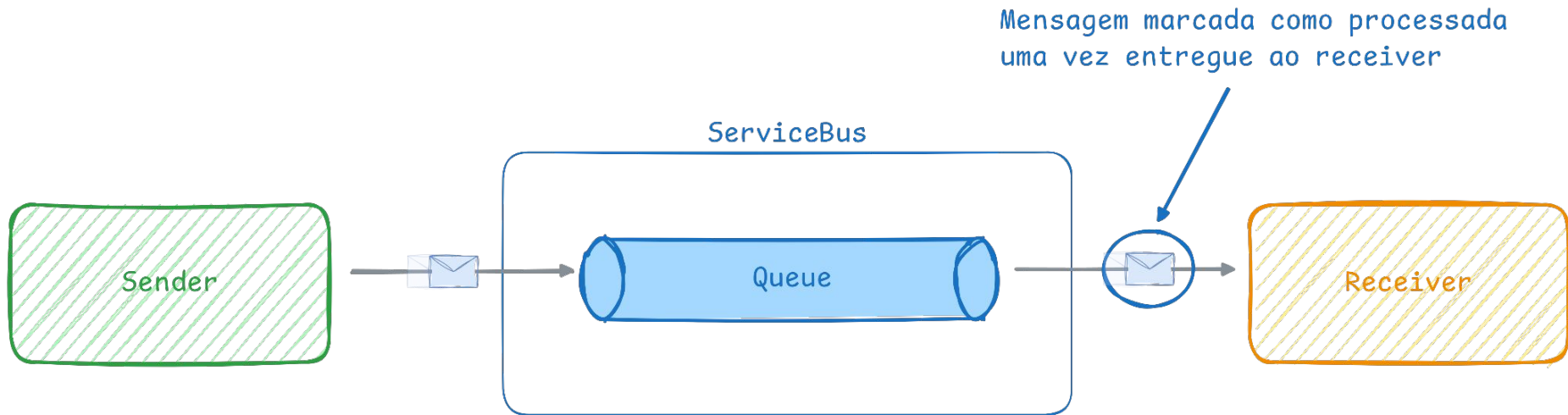
AZURE SERVICE BUS - RECEIVE E DELETE MODE

Receive and delete Mode



AZURE SERVICE BUS - RECEIVE E DELETE MODE

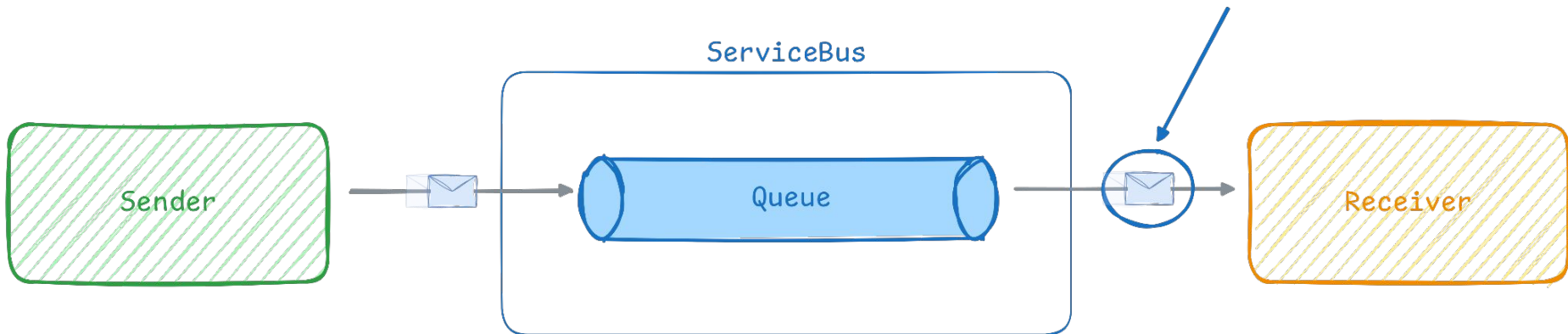
Receive and delete Mode



AZURE SERVICE BUS - RECEIVE E DELETE MODE

Receive and delete Mode

Garantia de entrega: At most once



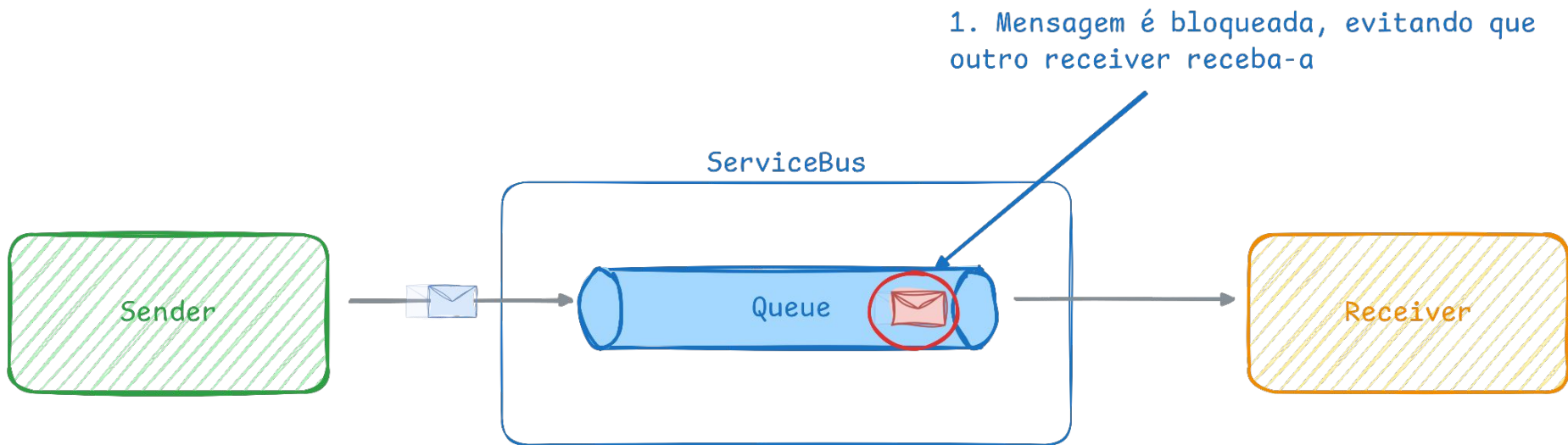
AZURE SERVICE BUS - PEEKLOCK

PeekLock



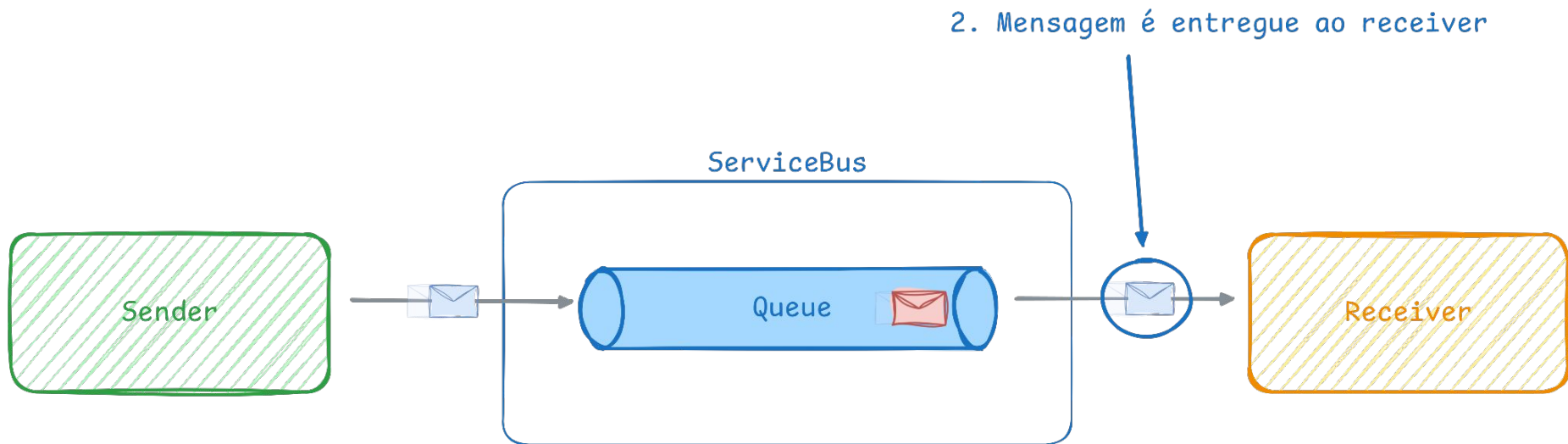
AZURE SERVICE BUS - PEEKLOCK

PeekLock



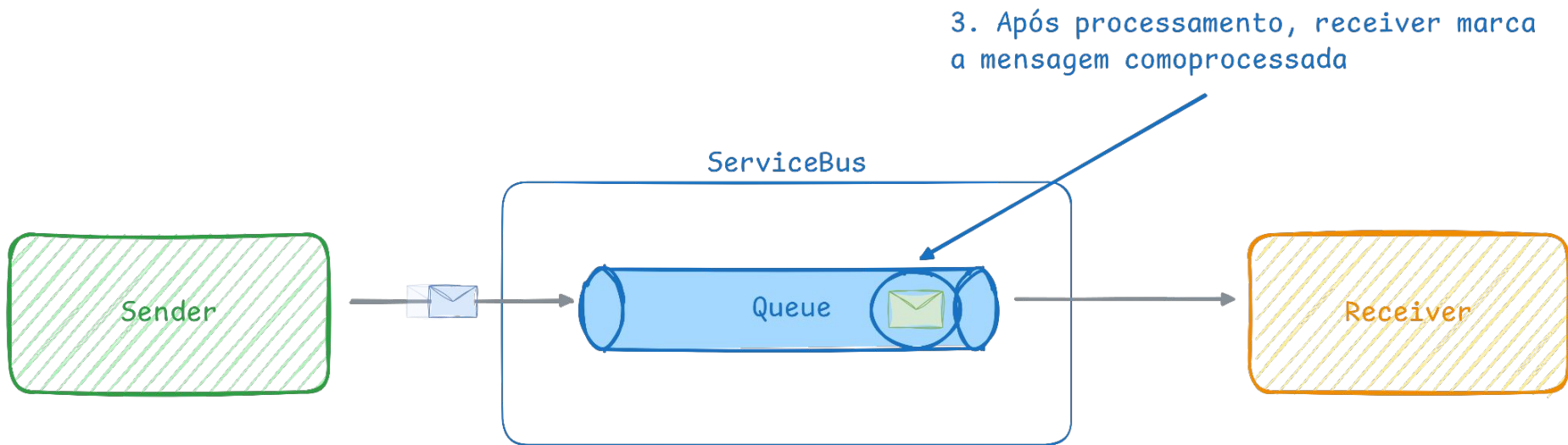
AZURE SERVICE BUS - PEEKLOCK

PeekLock



AZURE SERVICE BUS - PEEKLOCK

PeekLock

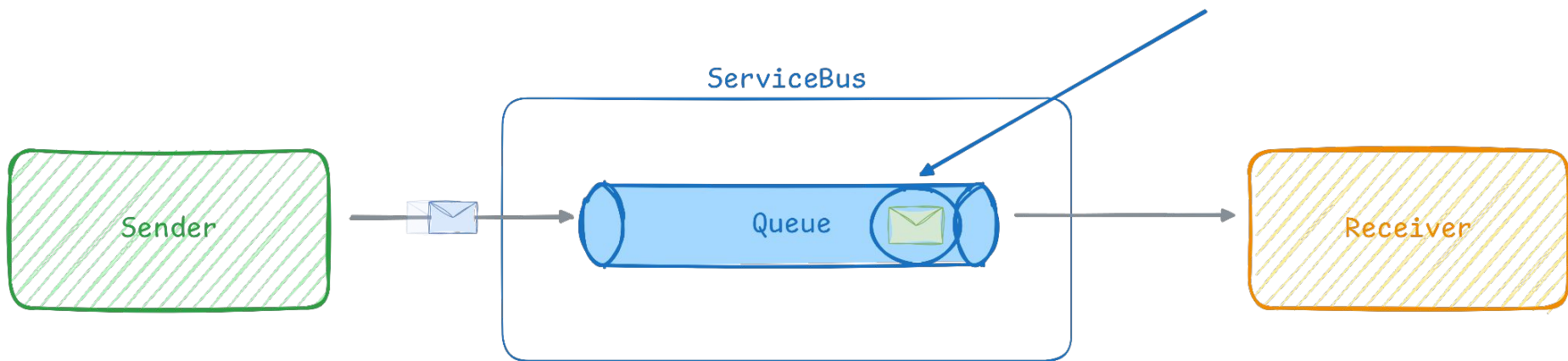


AZURE SERVICE BUS - PEEKLOCK

PeekLock

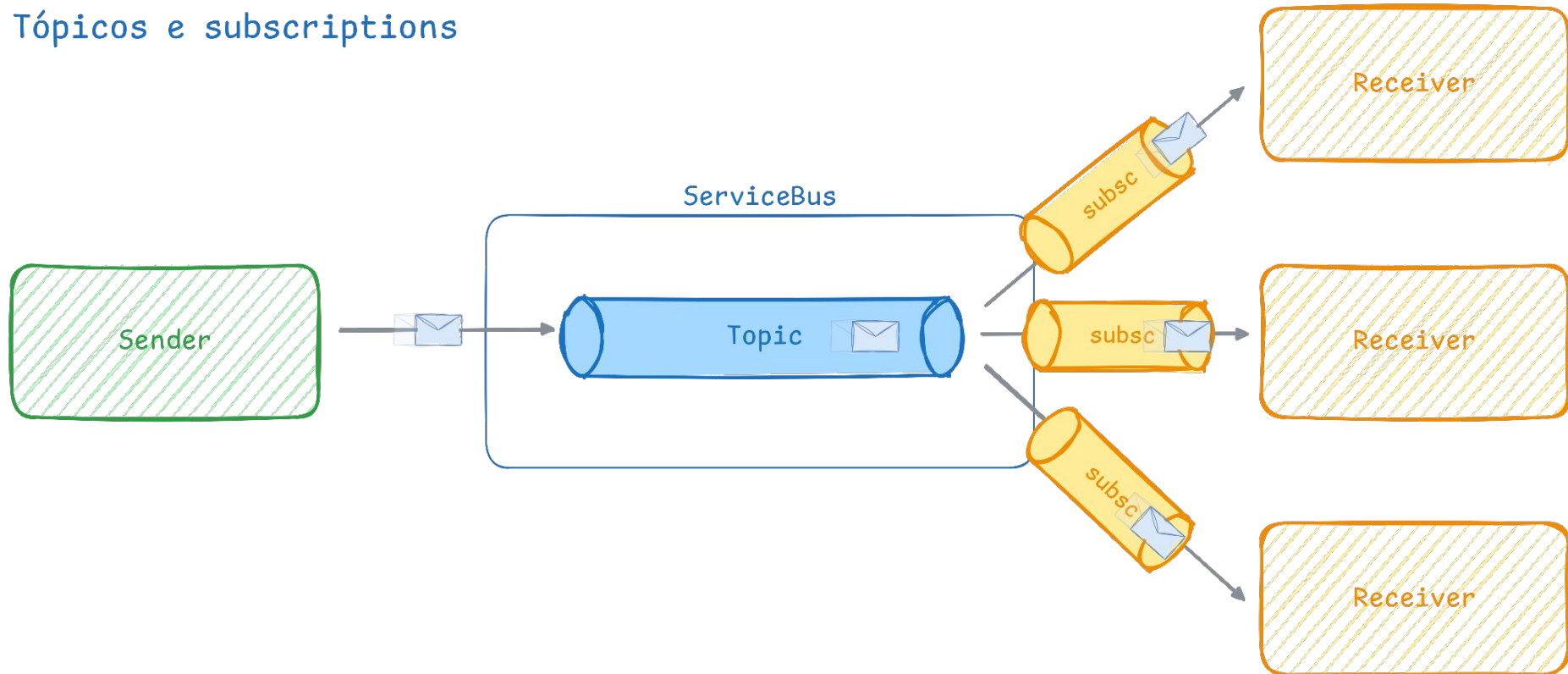
Garantia de entrega: At least once

3. Após processamento, receiver marca a mensagem como processada



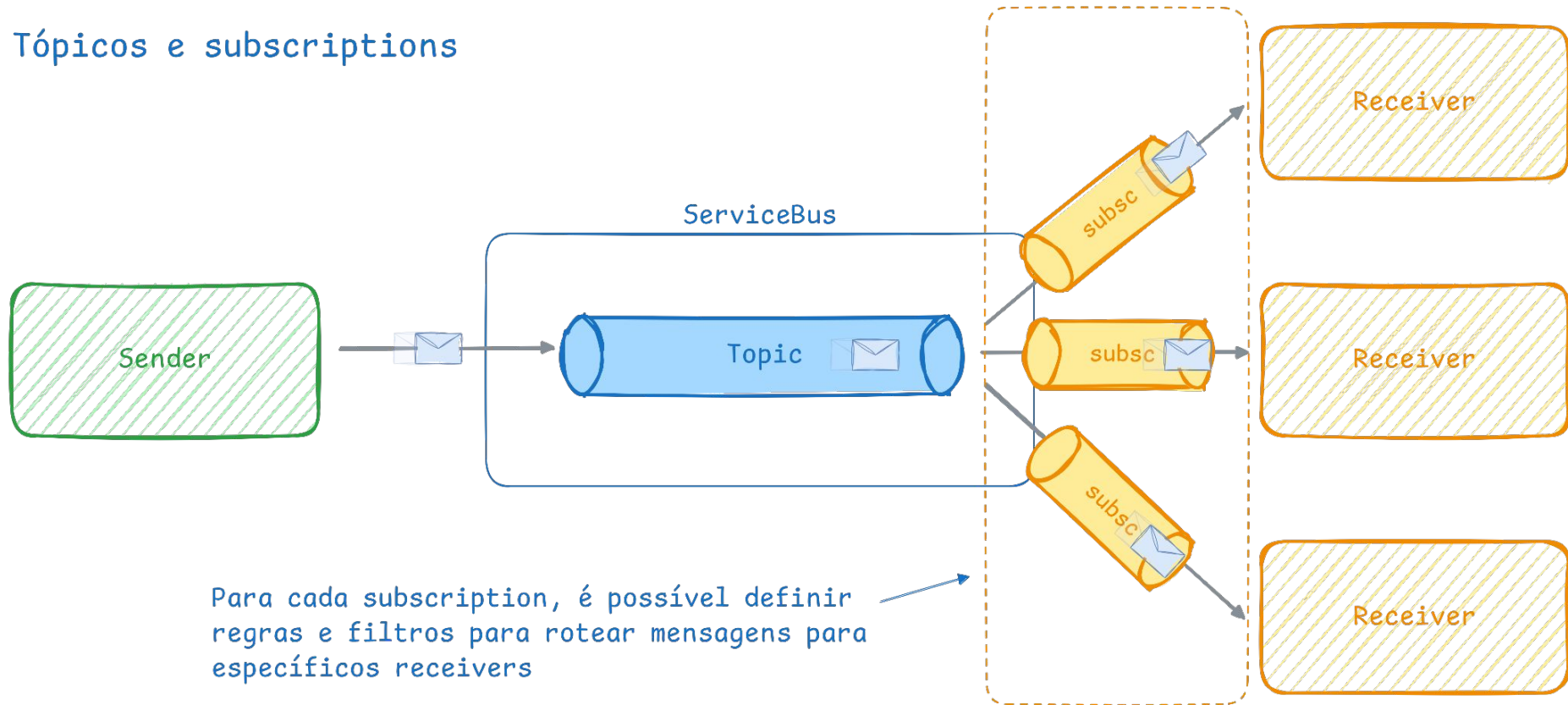
AZURE SERVICE BUS - TÓPICOS E SUBSCRIPTIONS

Tópicos e subscriptions



AZURE SERVICE BUS - TÓPICOS E SUBSCRIPTIONS

Tópicos e subscriptions





KAFKA - CARACTERÍSTICAS

- Alta performance, capacidade de processar milhões de mensagens por segundo
- Durabilidade das mensagens
- Escalabilidade horizontal
- Melhor escolha para cenários com grandes volumes de dados
- Suporte a message replay
- Suporta tanto o modelo **queue** quanto **publish-subscribe**

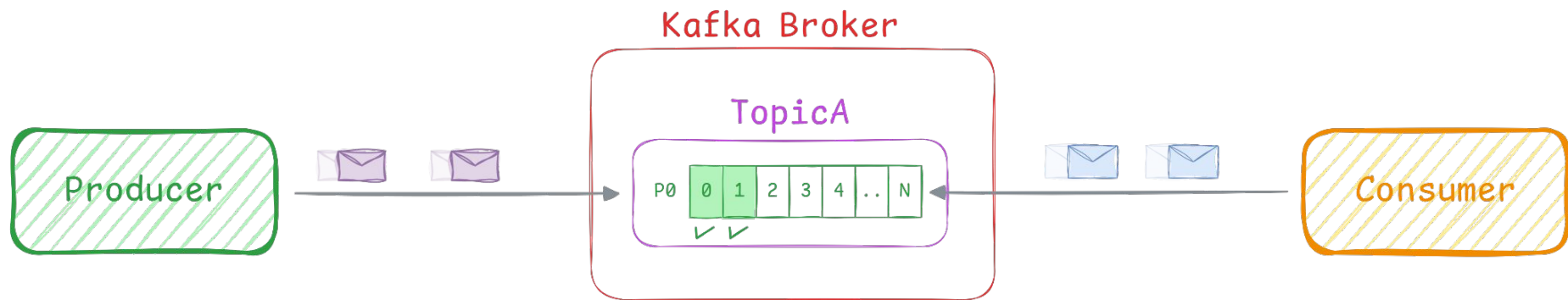
KAFKA - COMPONENTES

Topic

ConsumerGroup

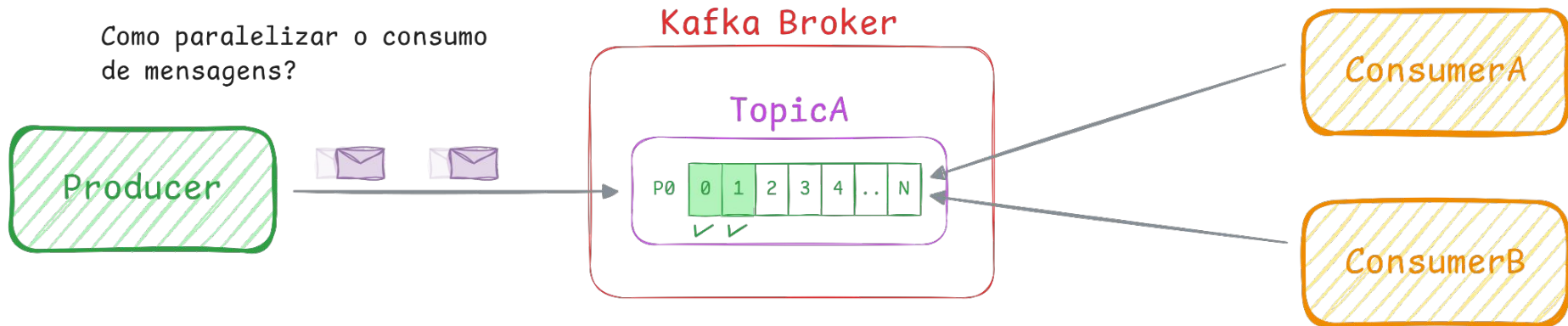
Partition

KAFKA - ARQUITETURA



KAFKA - ARQUITETURA

Como paralelizar o consumo de mensagens?

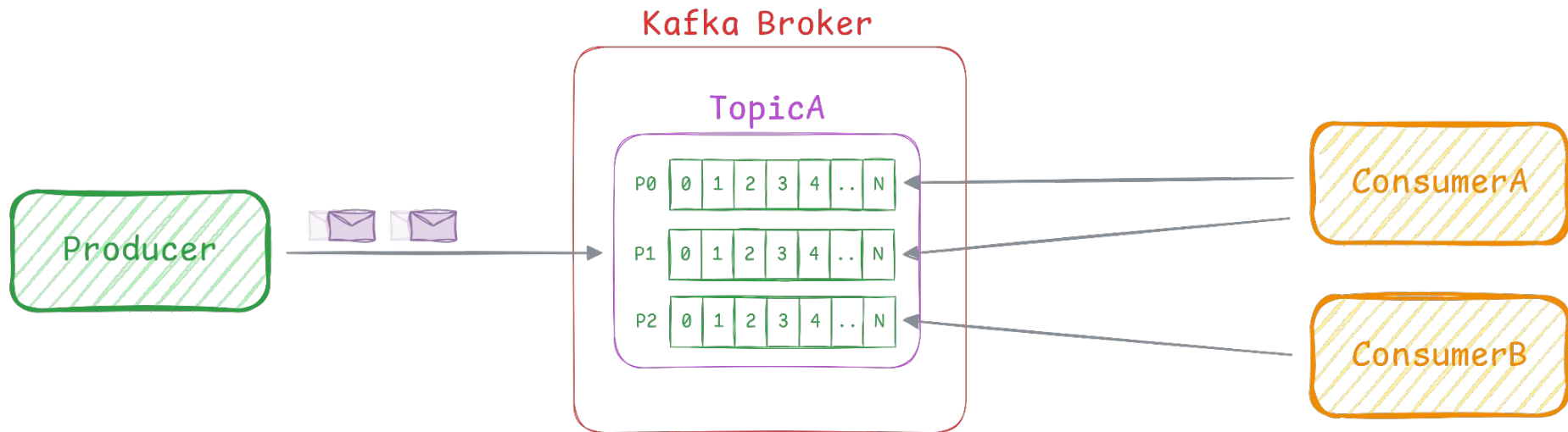


KAFKA - ARQUITETURA

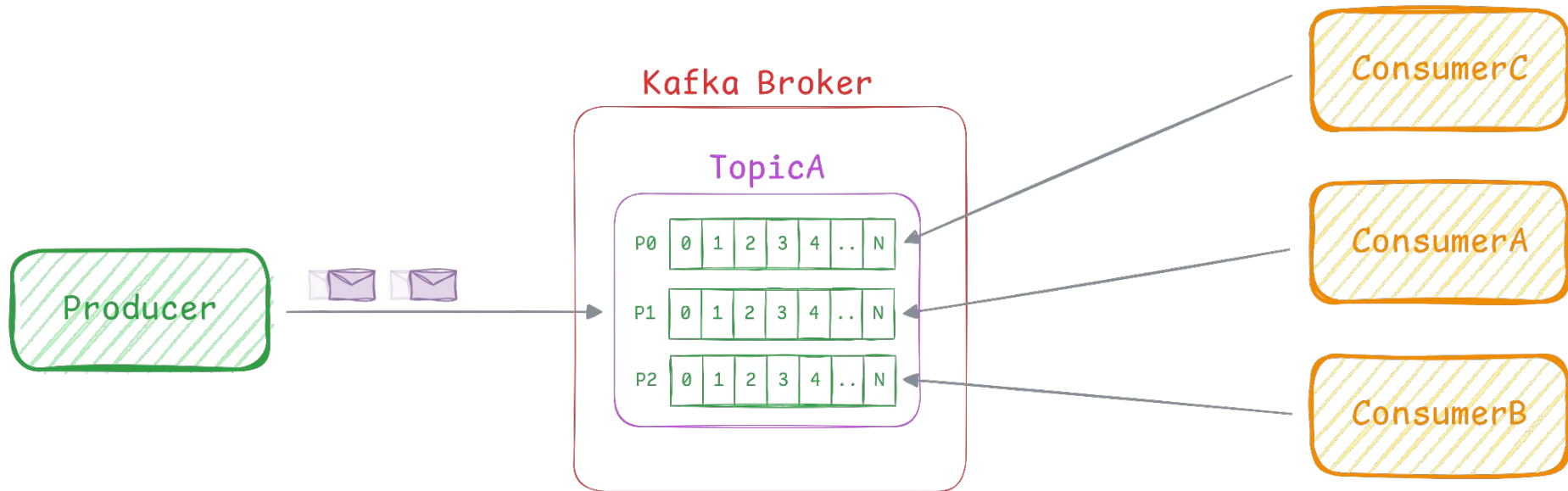


Esse consumidor não vai receber nenhuma mensagem,
pois a partition 0 já está atribuída ao consumerA

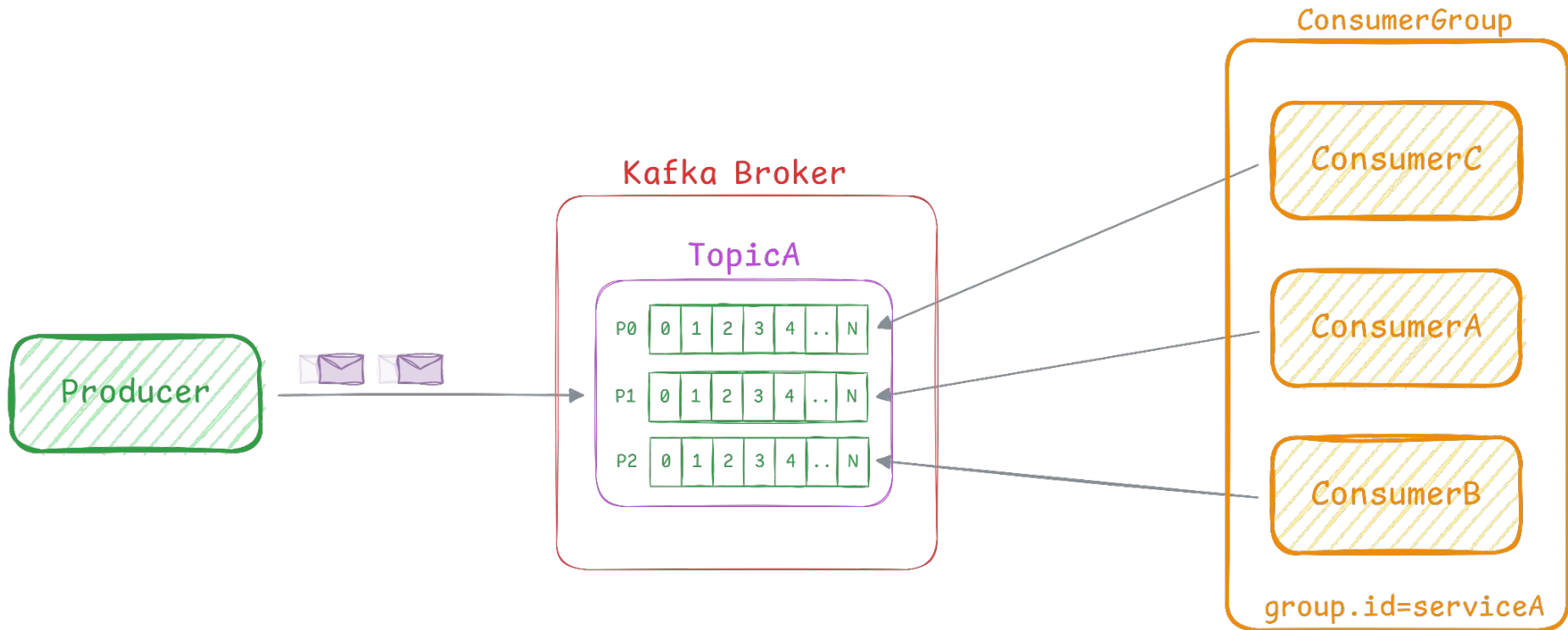
KAFKA - ARQUITETURA



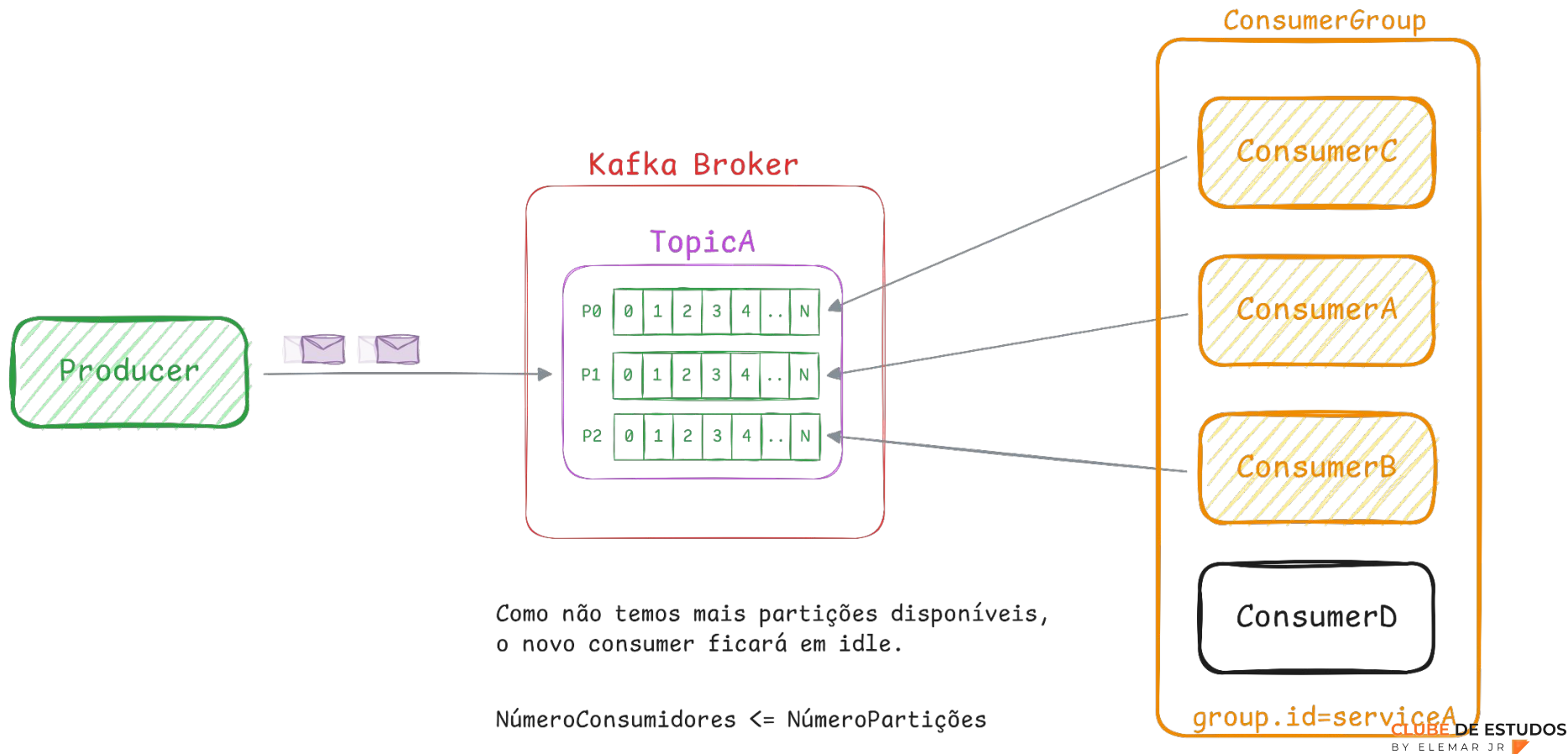
KAFKA - ARQUITETURA



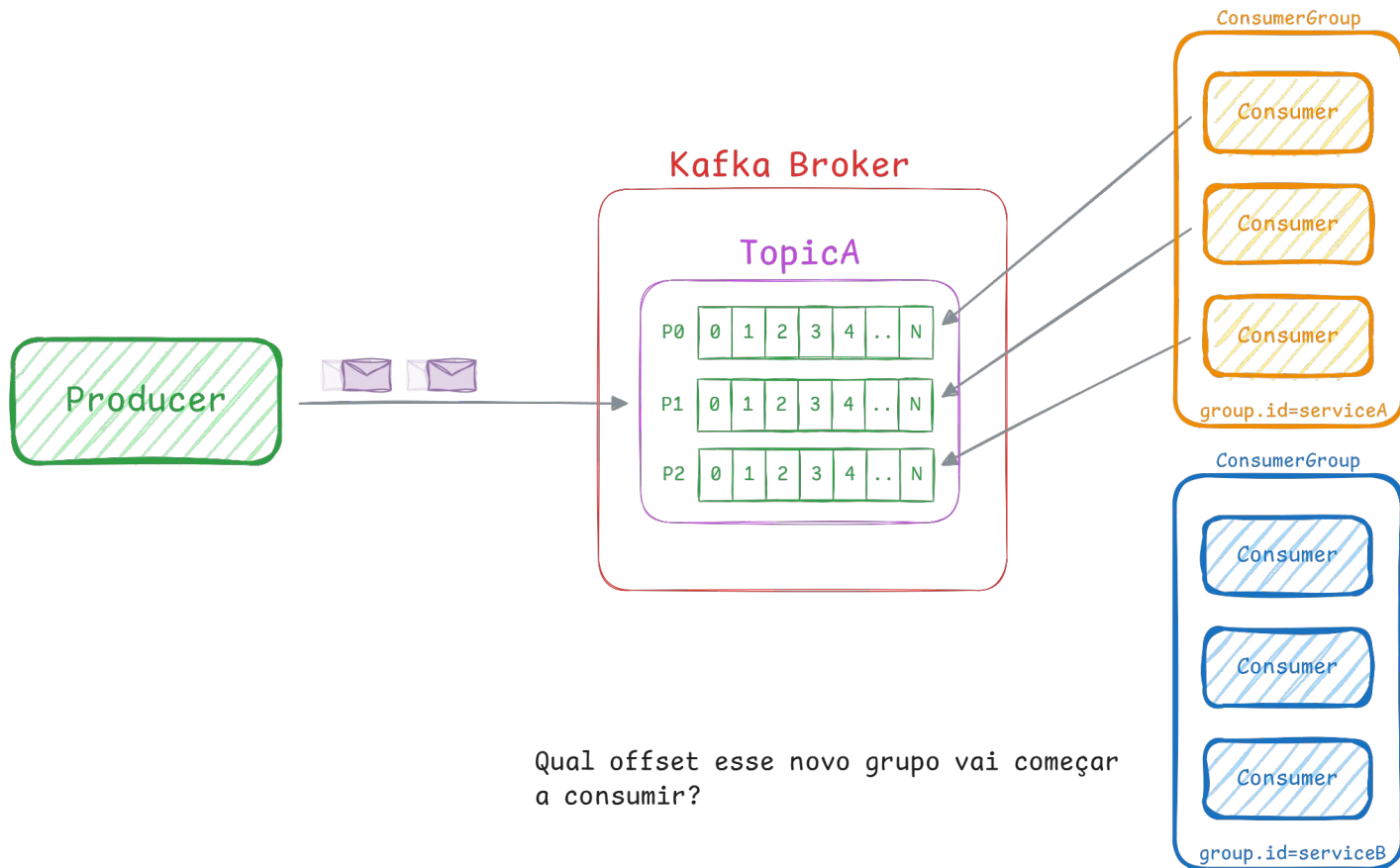
KAFKA - ARQUITETURA



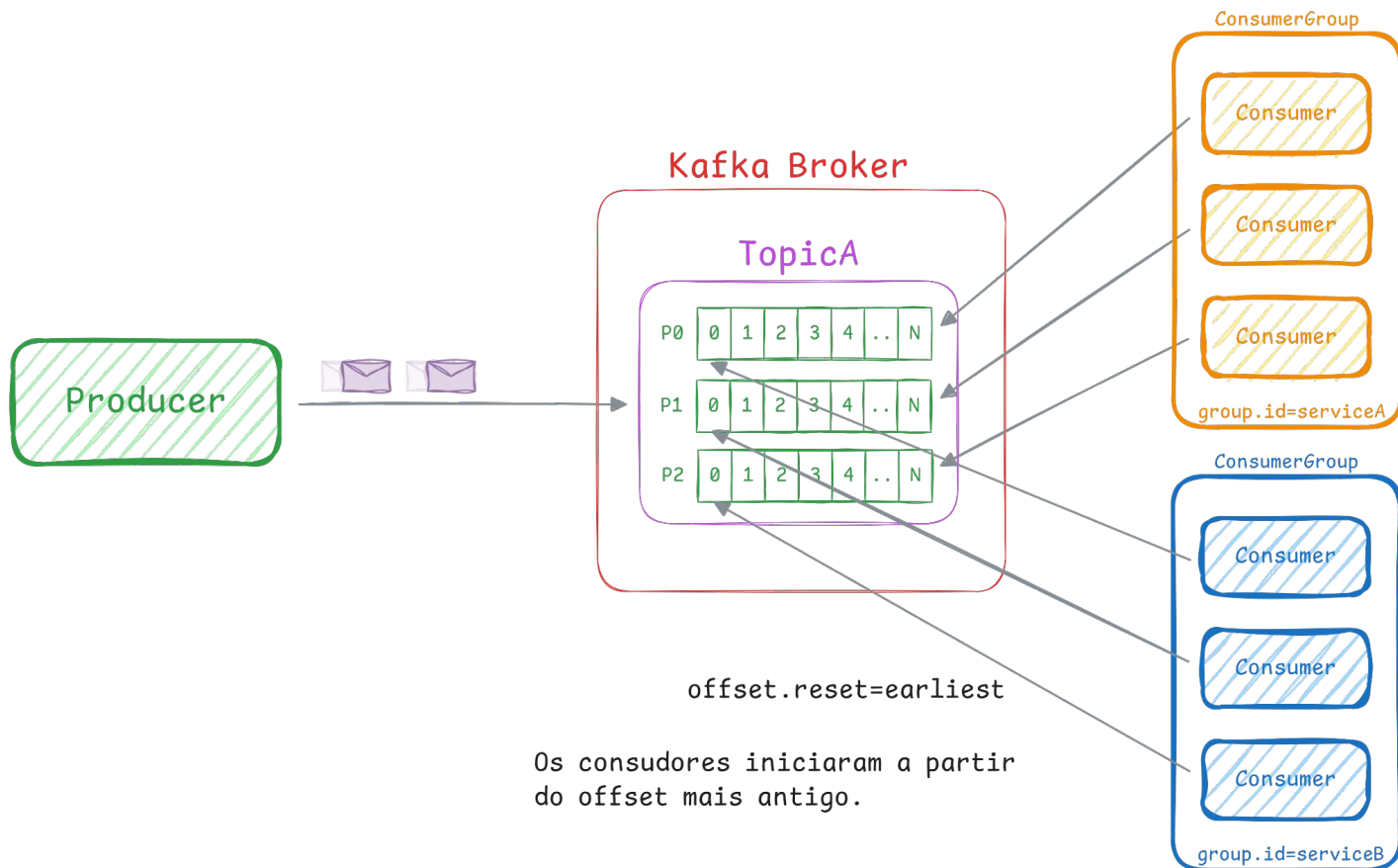
KAFKA - ARQUITETURA



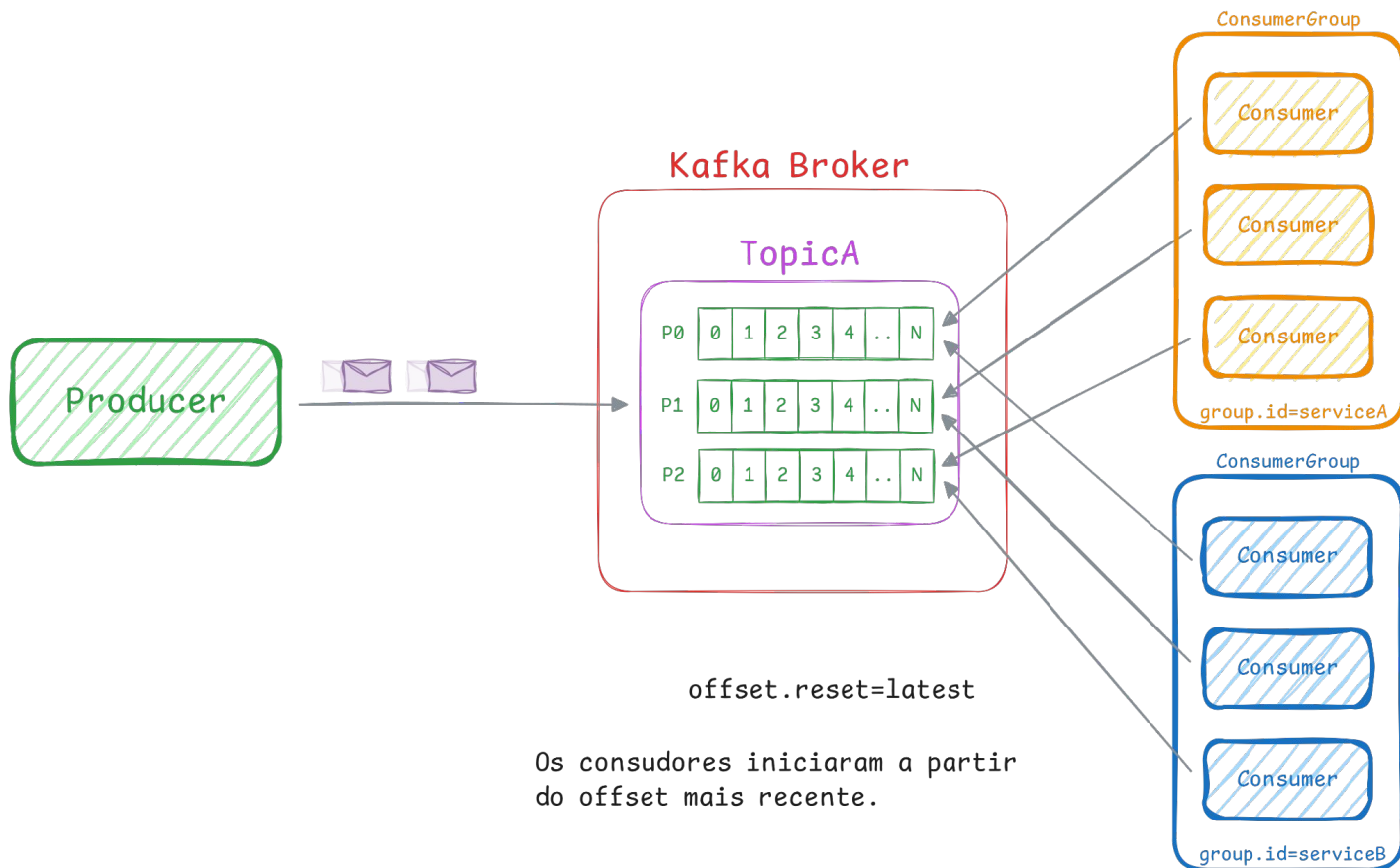
KAFKA - ARQUITETURA



KAFKA - ARQUITETURA



KAFKA - ARQUITETURA



PROJETANDO ARQUITETURAS ROBUSTAS COM KAFKA



KAFKA - ARQUITETURA

Throughput

Taxa processamento na produção e consumo

Confiabilidade

Integridade e consistência dos dados produzidos e consumidos

Disponibilidade

Ponto único de falha

Idempotência

Garantias que mensagens vão ser consumida apenas uma vez

KAFKA - CONFIGURAÇÕES IMPORTANTES

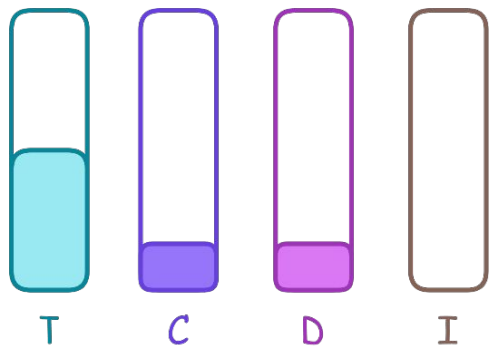
acks: Número de confirmações que o producer precisa receber para garantir que a mensagem foi entregue

min.insync.replicas: Número de réplicas que precisam confirmar que a entrega da mensagem foi feita com sucesso

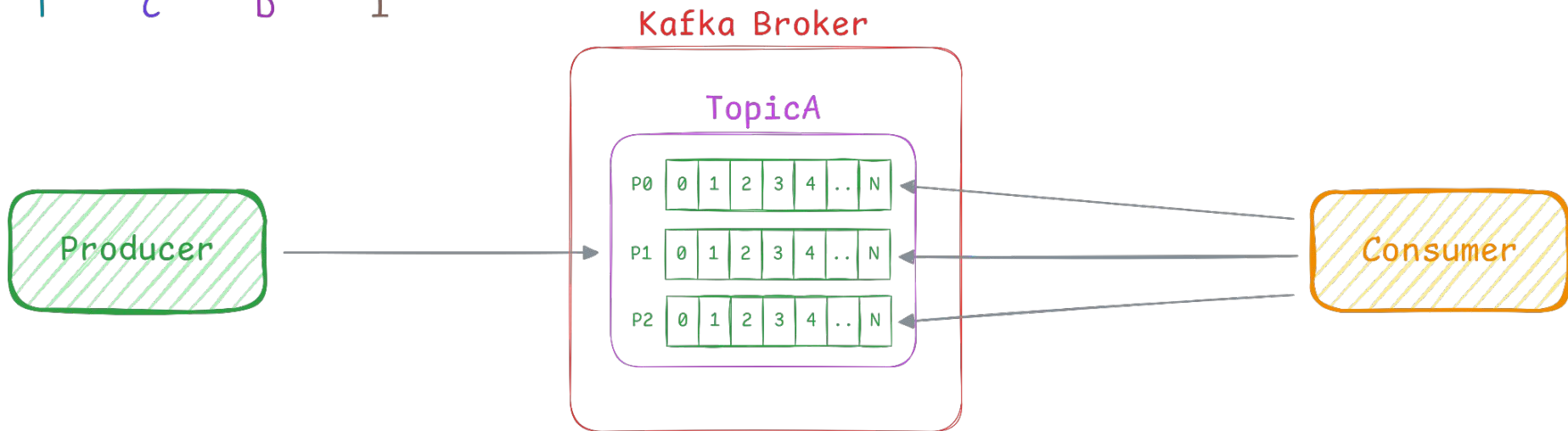
replication.factor: Número de réplicas da partição

enable.auto.commit: Determina se o offset das mensagens consumidas vai ser comitado de forma automática

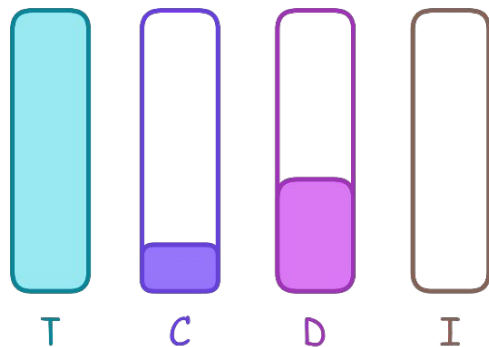
KAFKA - CENÁRIO 1



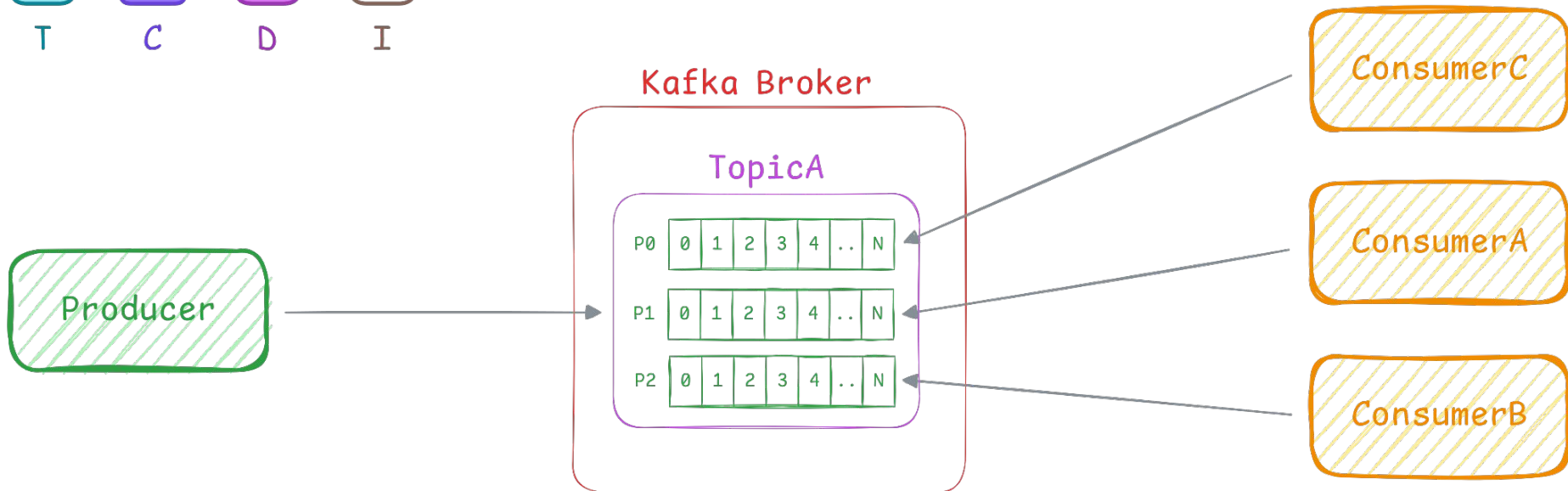
```
acks=0  
min.insync.replica=1  
replication.factor=1  
enable.auto.commit=true
```



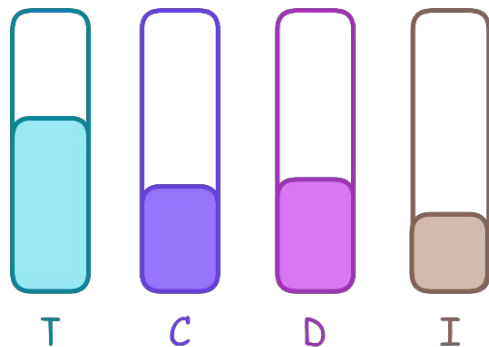
KAFKA - CENÁRIO 2



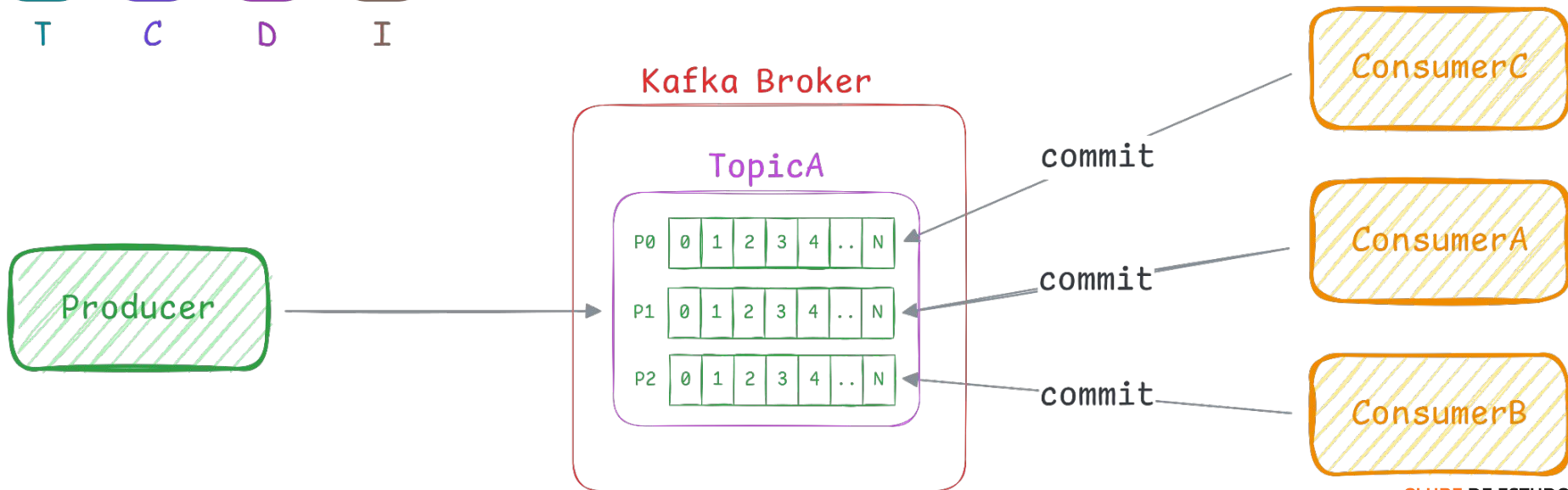
```
acks=0  
min.insync.replica=1  
replication.factory=1  
enable.auto.commit=true
```



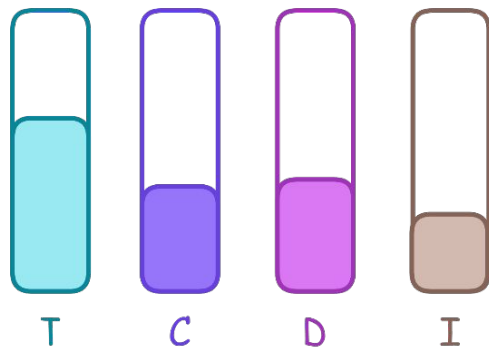
KAFKA - CENÁRIO 3



```
acks=0  
min.insync.replica=1  
replication.factor=1  
enable.auto.commit=false
```

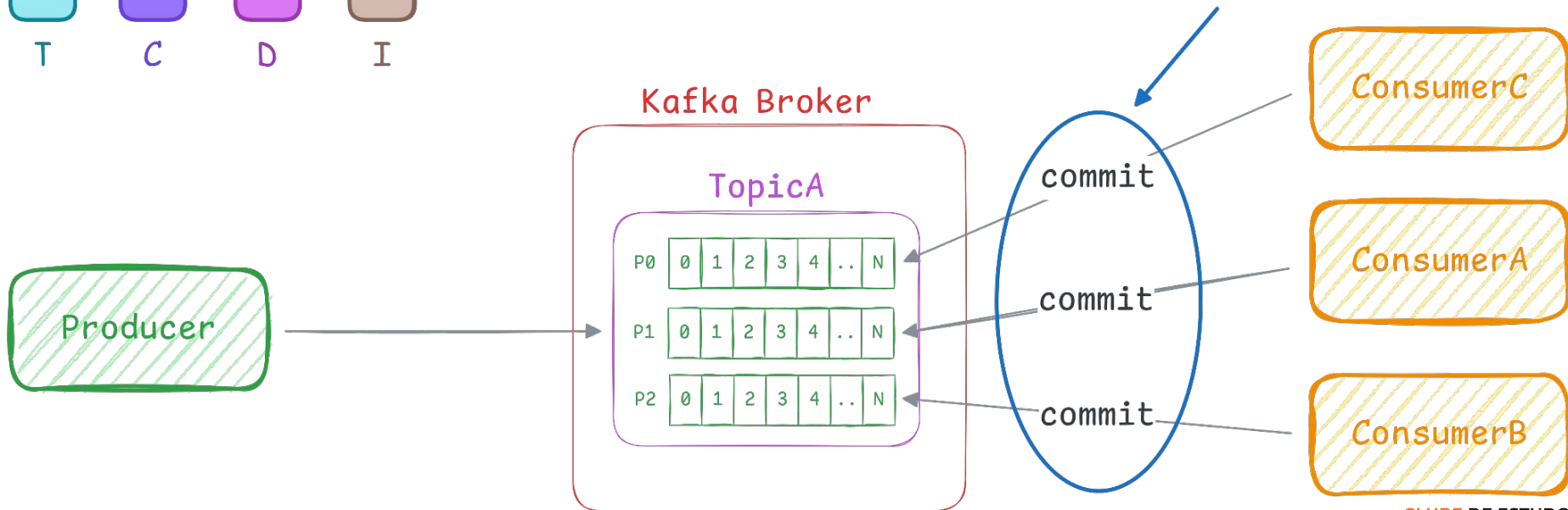


KAFKA - CENÁRIO 3

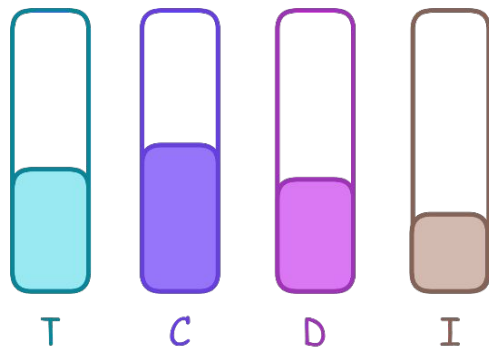


```
acks=0  
min.insync.replica=1  
replication.factor=1  
enable.auto.commit=false
```

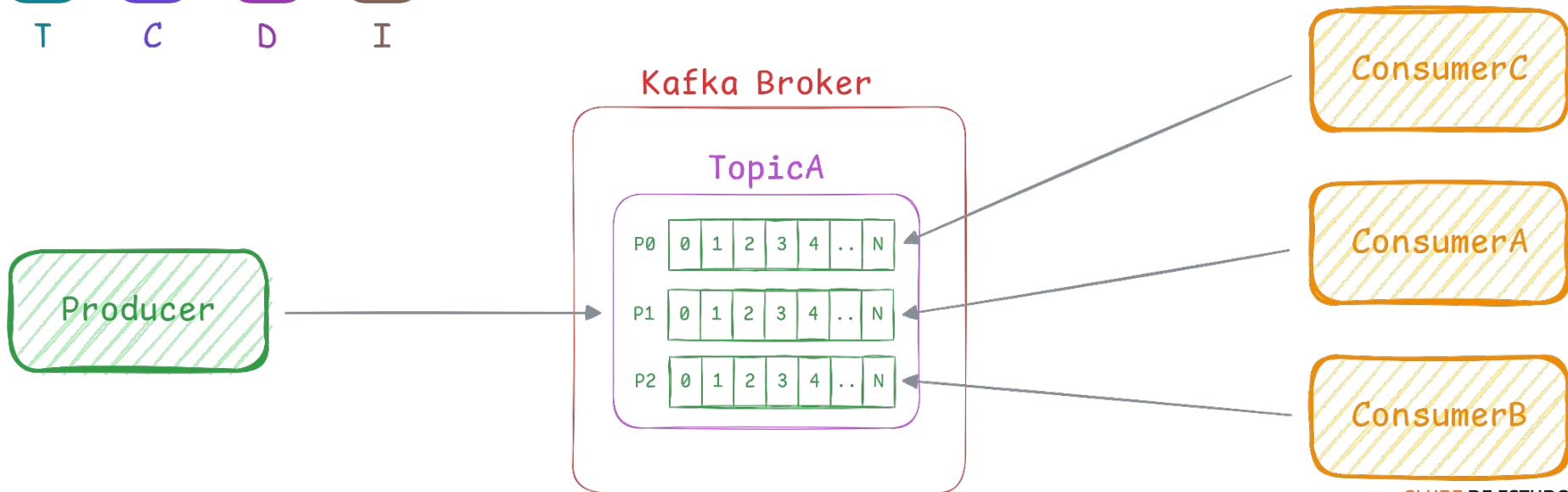
Commit manual gera em um tempo adicional no consumo das mensagens, principalmente em cenários de alto volume



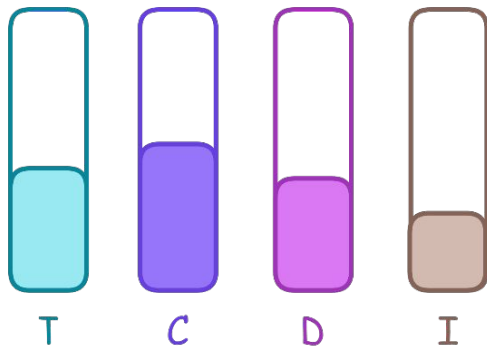
KAFKA - CENÁRIO 4



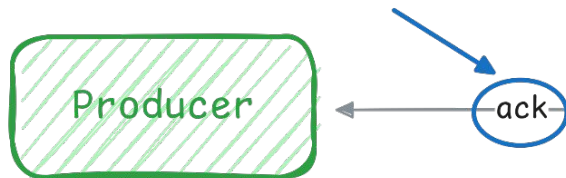
```
acks=1  
min.insync.replica=1  
replication.factor=1  
enable.auto.commit=false
```



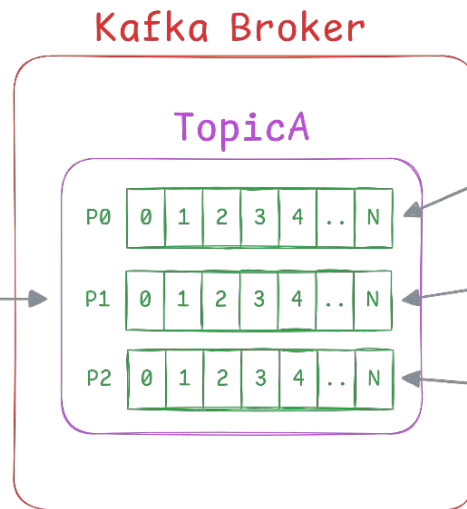
KAFKA - CENÁRIO 4



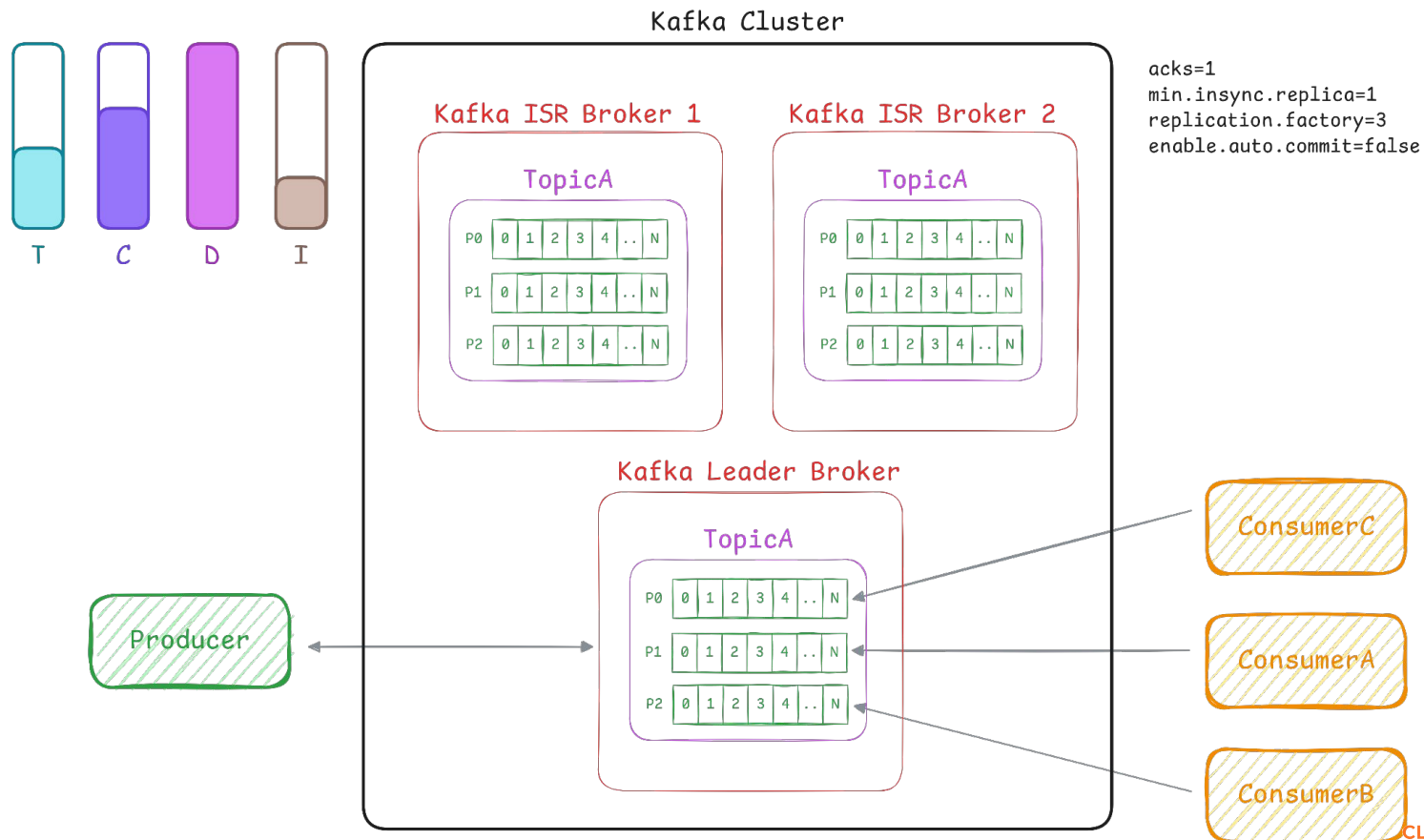
Producer aguarda uma confirmação que a mensagem foi recebida, impactando a taxa de produção de mensagens



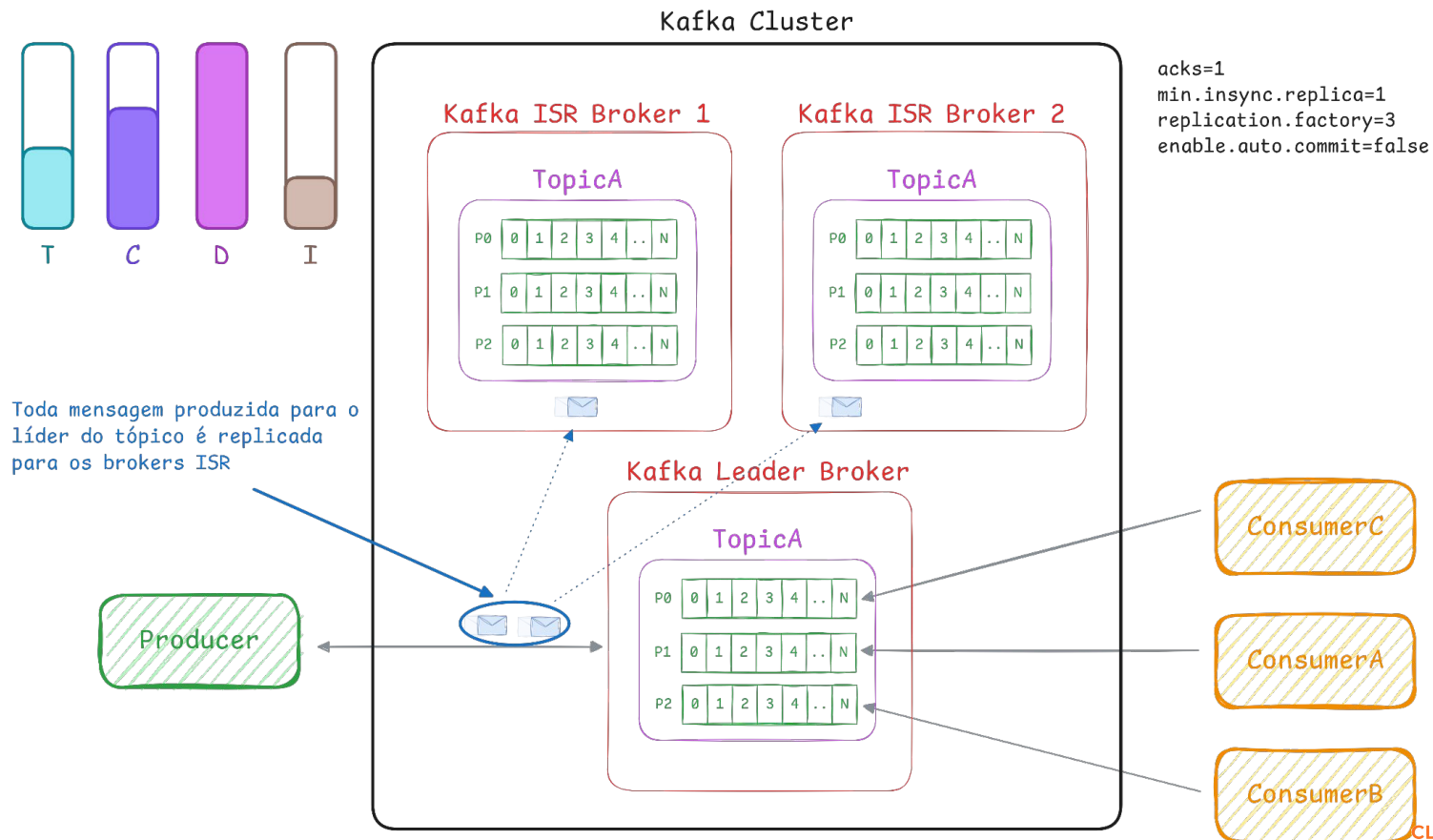
```
acks=1  
min.insync.replica=1  
replication.factor=1  
enable.auto.commit=false
```



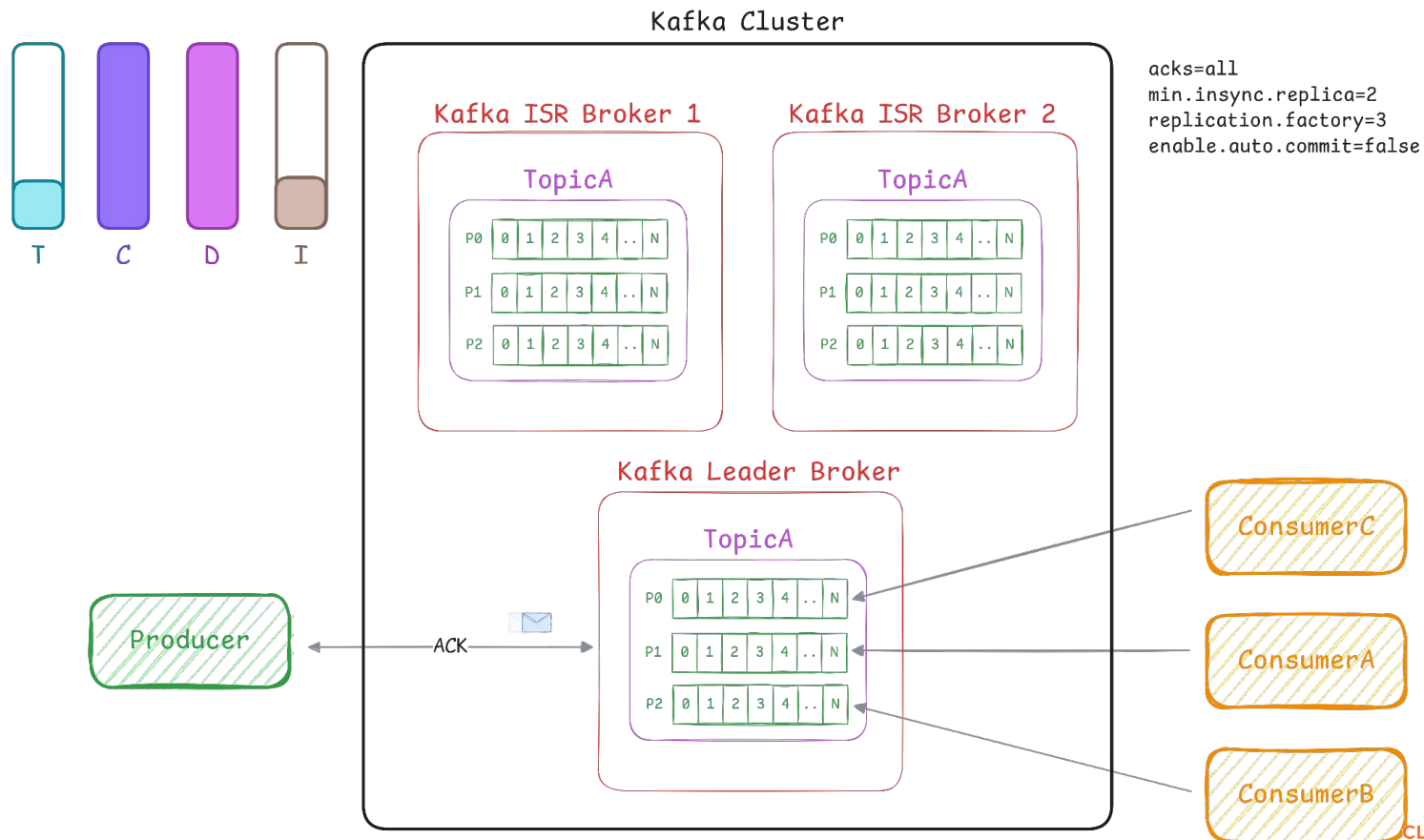
KAFKA - CENÁRIO 5



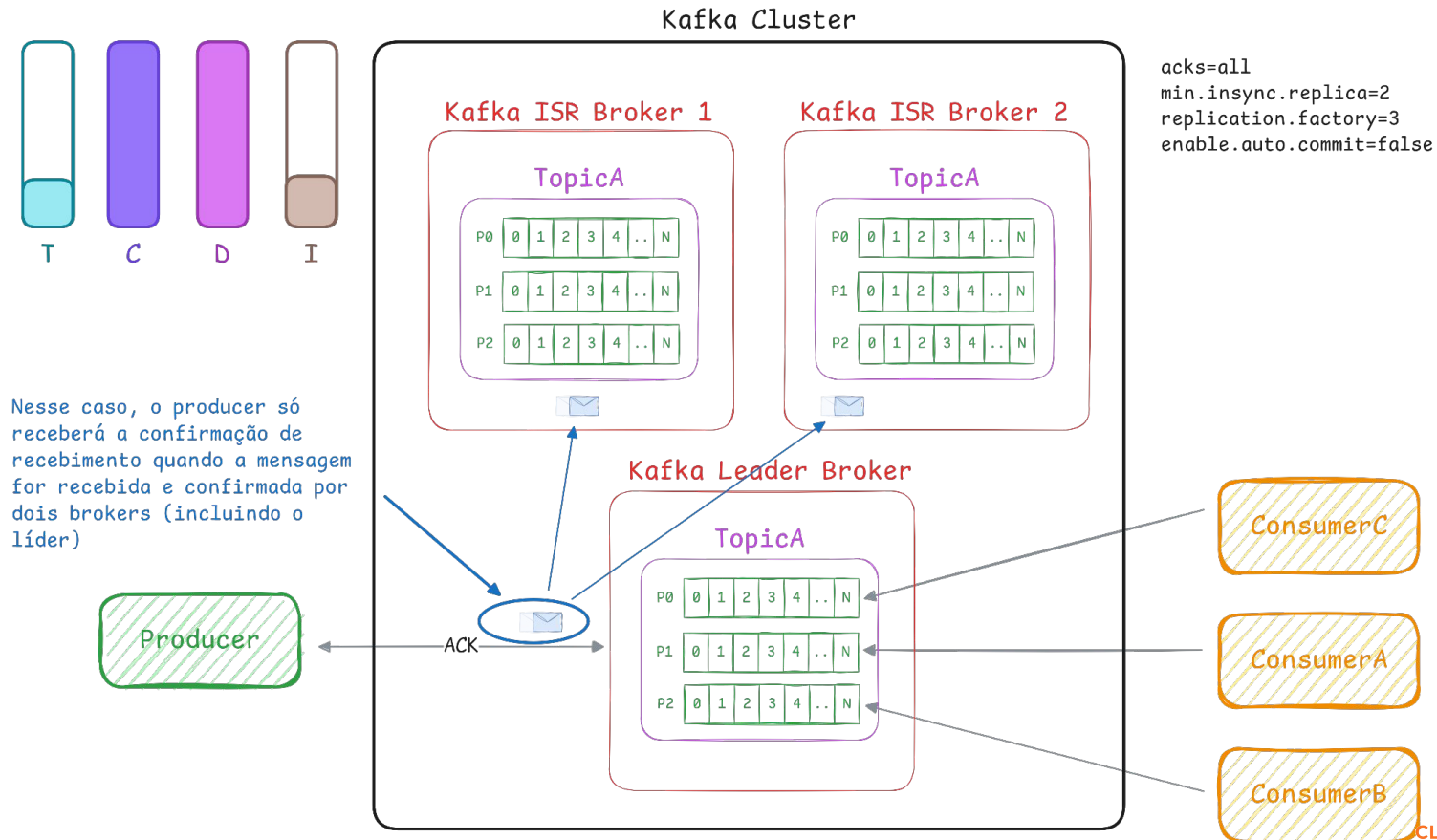
KAFKA - CENÁRIO 5



KAFKA - CENÁRIO 6



KAFKA - CENÁRIO 6



CLUBE DE ESTUDOS
BY ELEMAR JR 

ELEMAR